



Network and Cyber Security (AID4PM04A)

School of Computer Engineering and Technology (SCET)

B.Tech Final Year AIDS

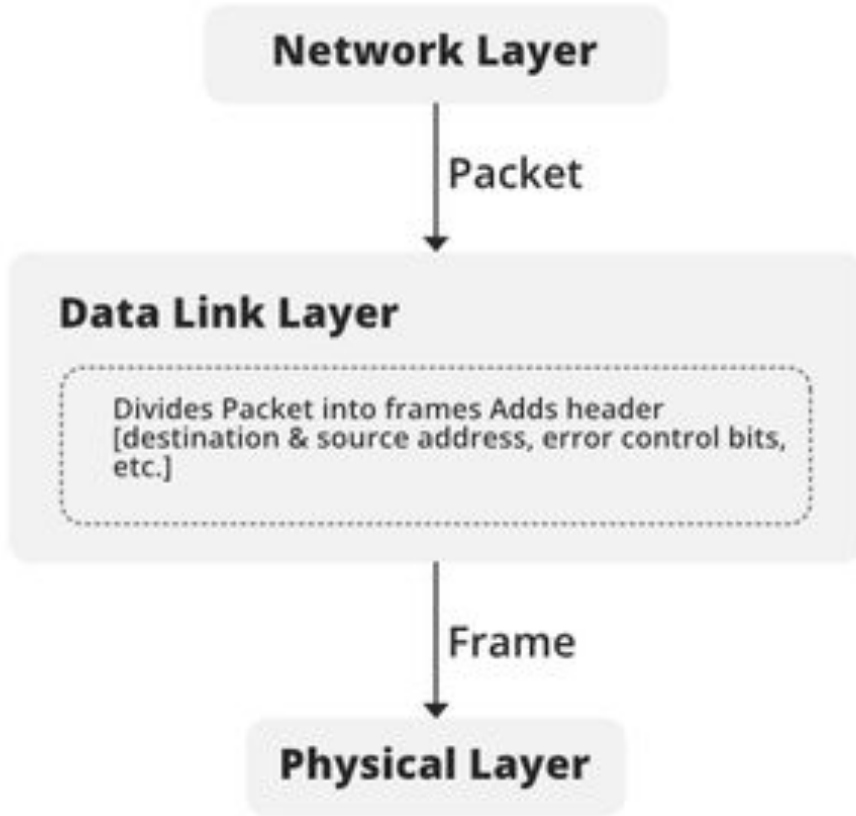
Unit 2: Data Link Layer

Data Link Layer

Contents:

Error Detection,
Error Correction,
Elementary Data Link Layer Protocols,
Sliding window Protocols: Stop and wait ARQ, Go Back N ARQ, Selective Repeat (SR) ,
Medium Access Layer,
Channel allocation Problem,
Multiple Access Protocols,
Network Layer Design Issues,
Routing Algorithms,
Congestion Control Algorithm

Data Link Layer



At the sender's side, DLL receives packets from the Network layer and divides them into small frames, then, sends each frame bit-by-bit to the physical layer.

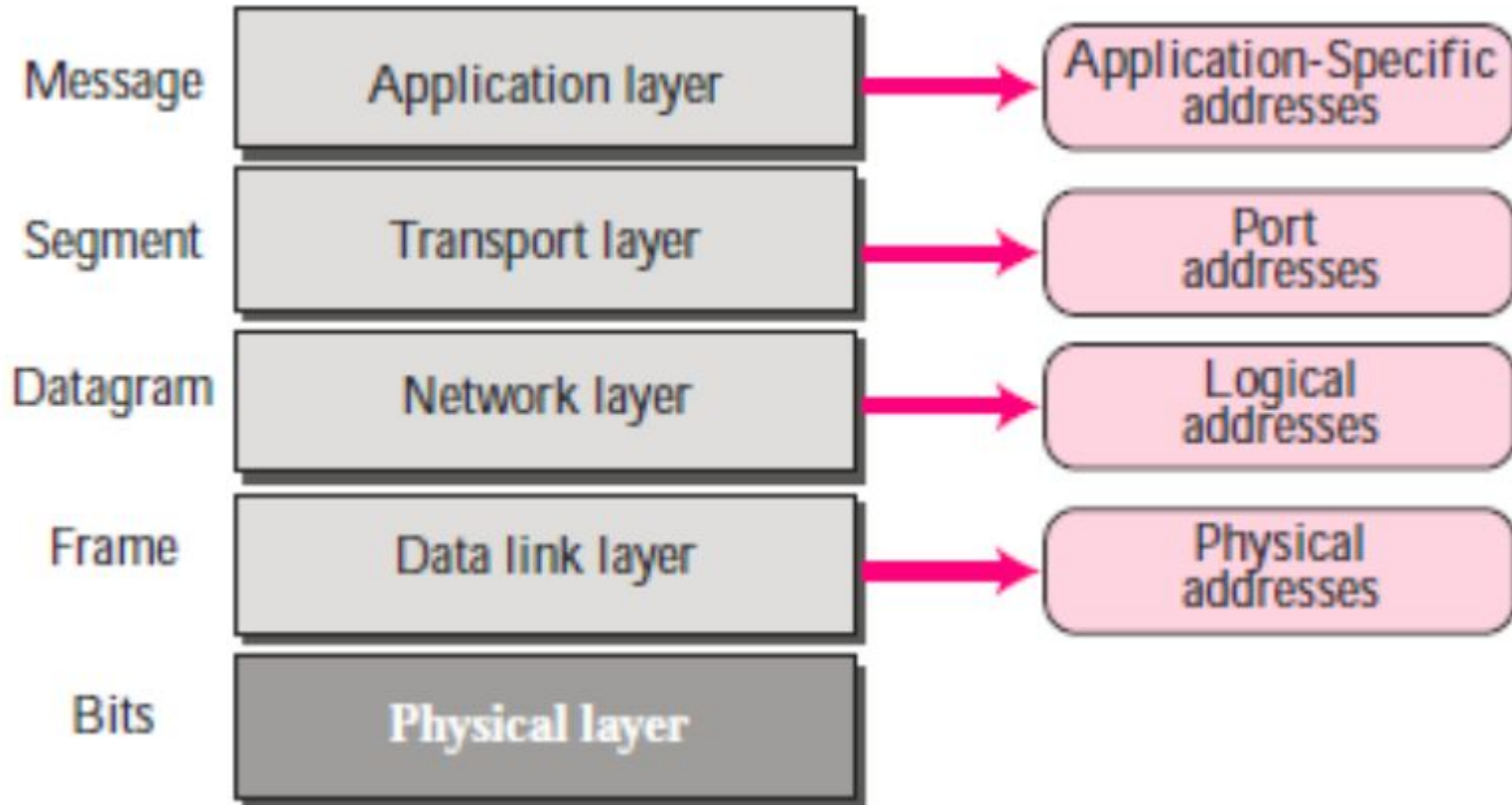
It also attaches some special bits (for error control and addressing) at the header and end of the frame.

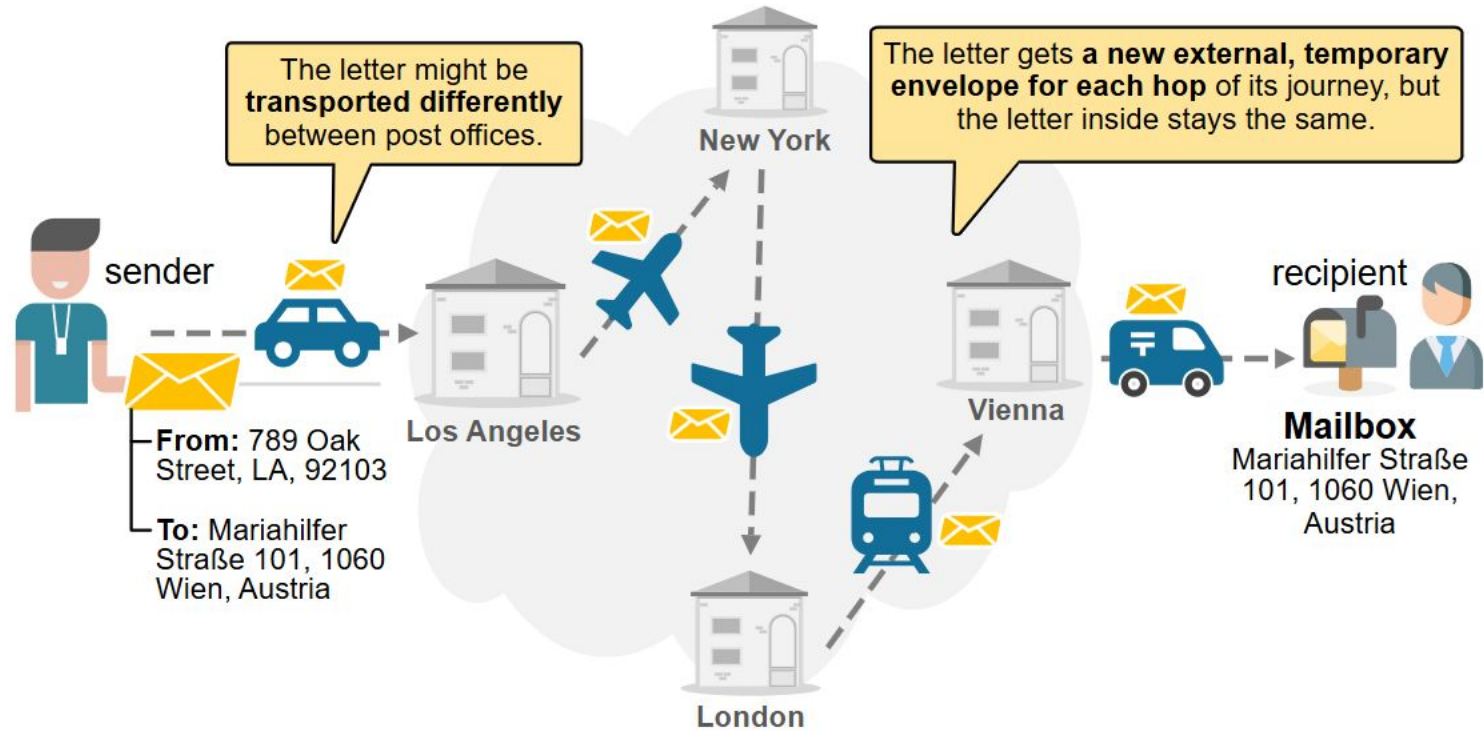
At the receiver's end, DLL takes bits from the Physical layer organizes them into the frame, and sends them to the Network layer.

Functions of Data Link Layer

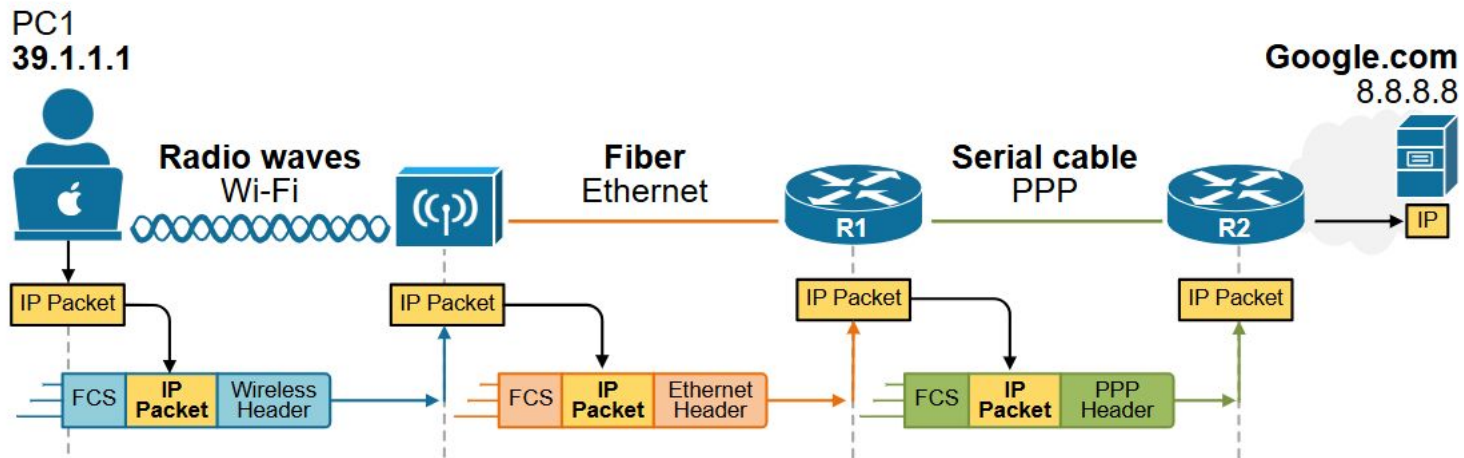
- 1.Framing:** Divides data into frames for transmission.
- 2.Physical (MAC) Addressing:** Uses MAC addresses to identify sender and receiver.
- 3.Error Detection:** Detects transmission errors using techniques like CRC (Cyclic Redundancy Check).
- 4.Flow Control:** Prevents a fast sender from overwhelming a slow receiver.
- 5.Access Control (Media Access Control):** Determines which device can transmit when multiple devices share the same medium.

Addresses in TCP/IP Network





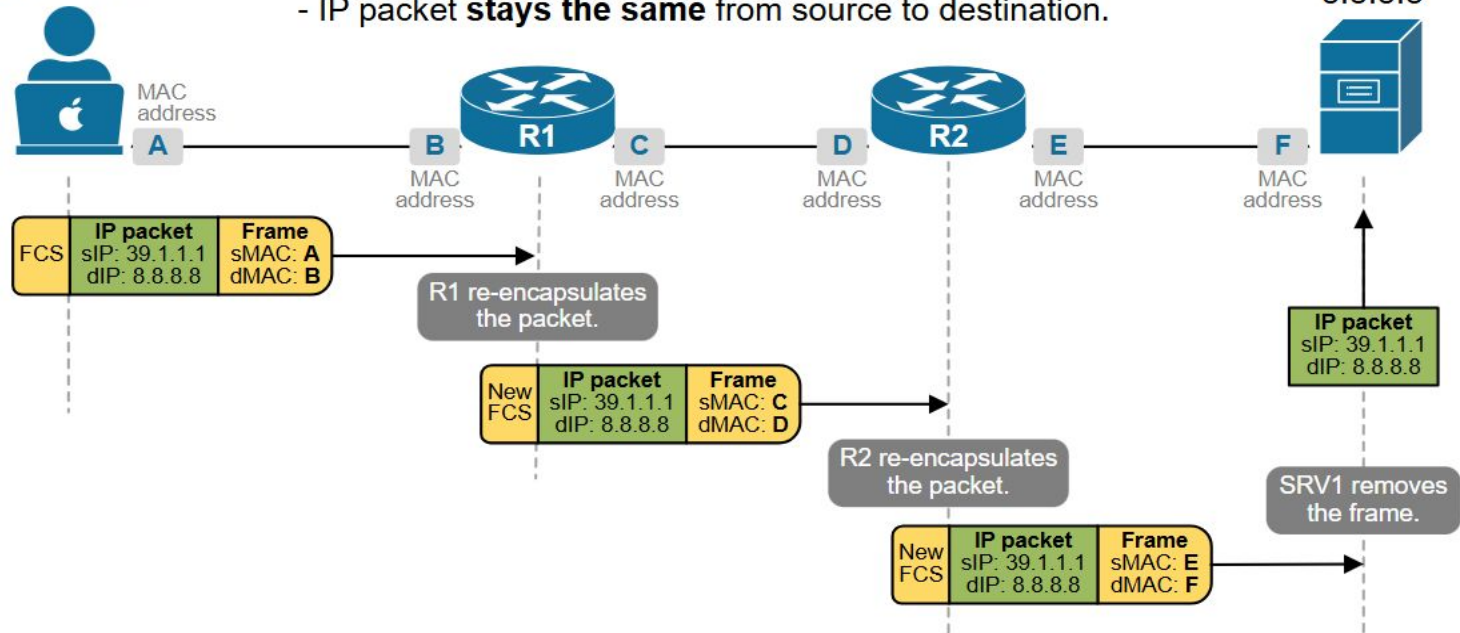
The Data Link Layer - Postal Service example.



Hop-by-hop frame re-encapsulation.

PC1
39.1.1.1

- Layer 2 frame is **removed and rebuilt** at each hop.
- IP packet **stays the same** from source to destination.



Frame hop-by-hop re-encapsulation.

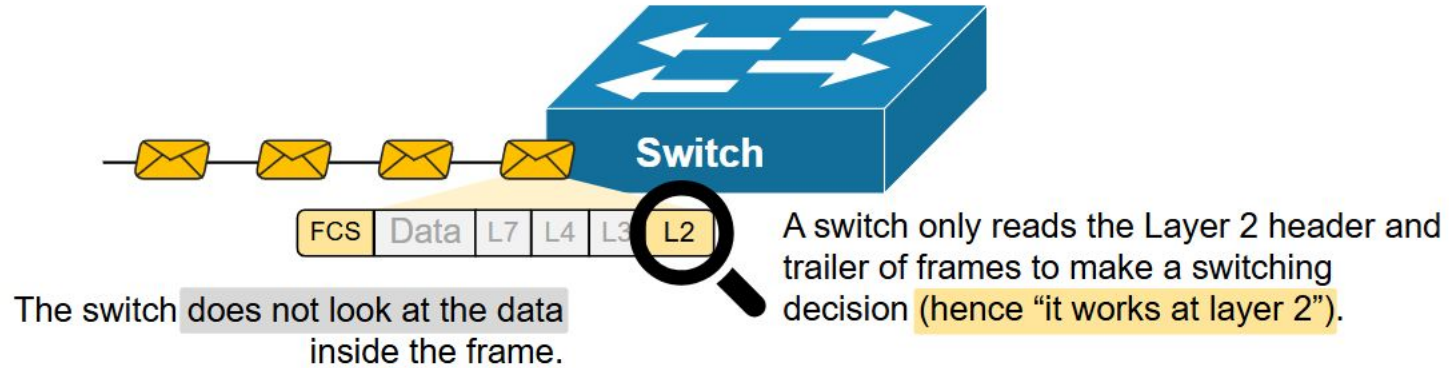
Sublayers in DLL

Logical Link Control (LLC)

- Manages communication between the Network Layer and MAC sublayer.
- Provides flow and error control.

Media Access Control (MAC)

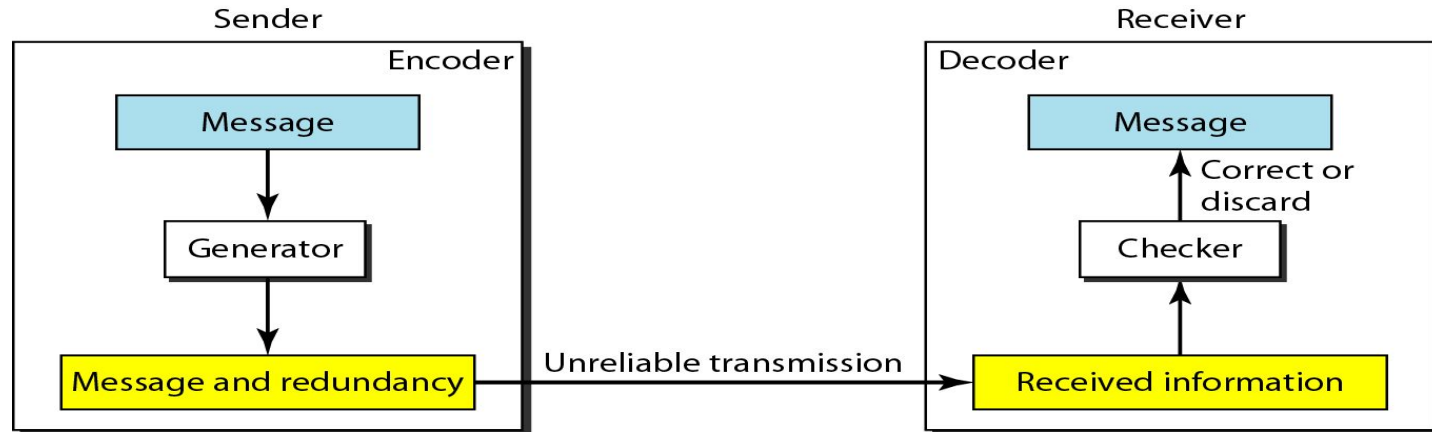
- Controls access to the transmission medium.
- Handles MAC addressing.



A switch operates at Layer 2.

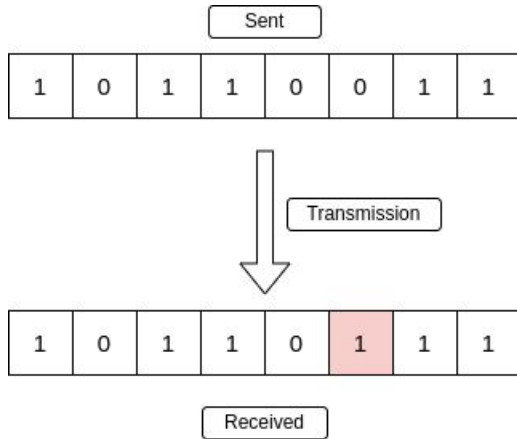
Error Control

- To detect or correct errors, we need to send extra (redundant) bits with data.
- Enough redundancy is added to detect an error.
- In error detection, the receiver knows an error occurred but does not know which bit(s) is(are) in error.
- Error detection has less overhead than error correction.

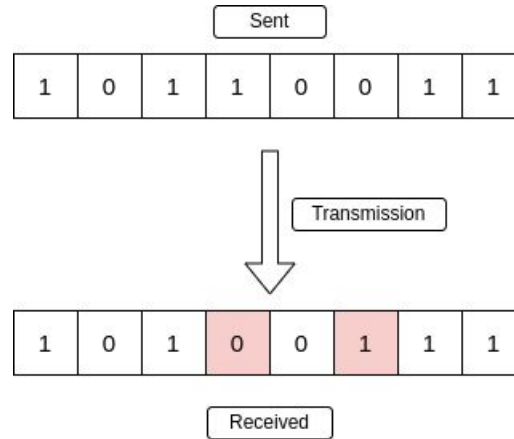


Types of Error

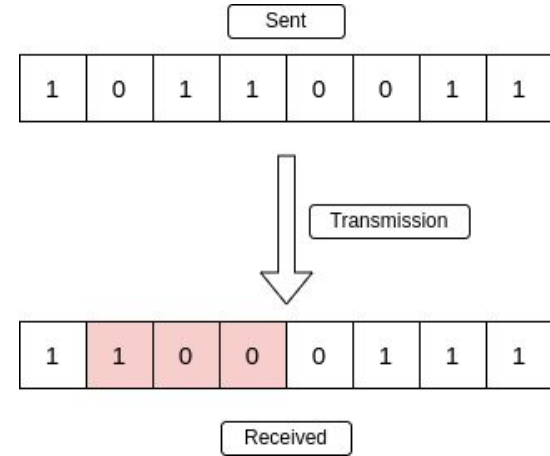
Single-Bit Error



Multiple-Bit Error



Burst Error



Error Detection and Correction Codes

- During transmission of binary data from one system to the other, the noise may also be added. Due to this, there may be errors in the received data at other system.
- That means a bit 0 may change to 1 or a bit 1 may change to 0. We can't avoid the interference of noise.
- The original data can be recovered first by detecting whether any error is present and then correcting those errors

For this purpose, following codes are used.

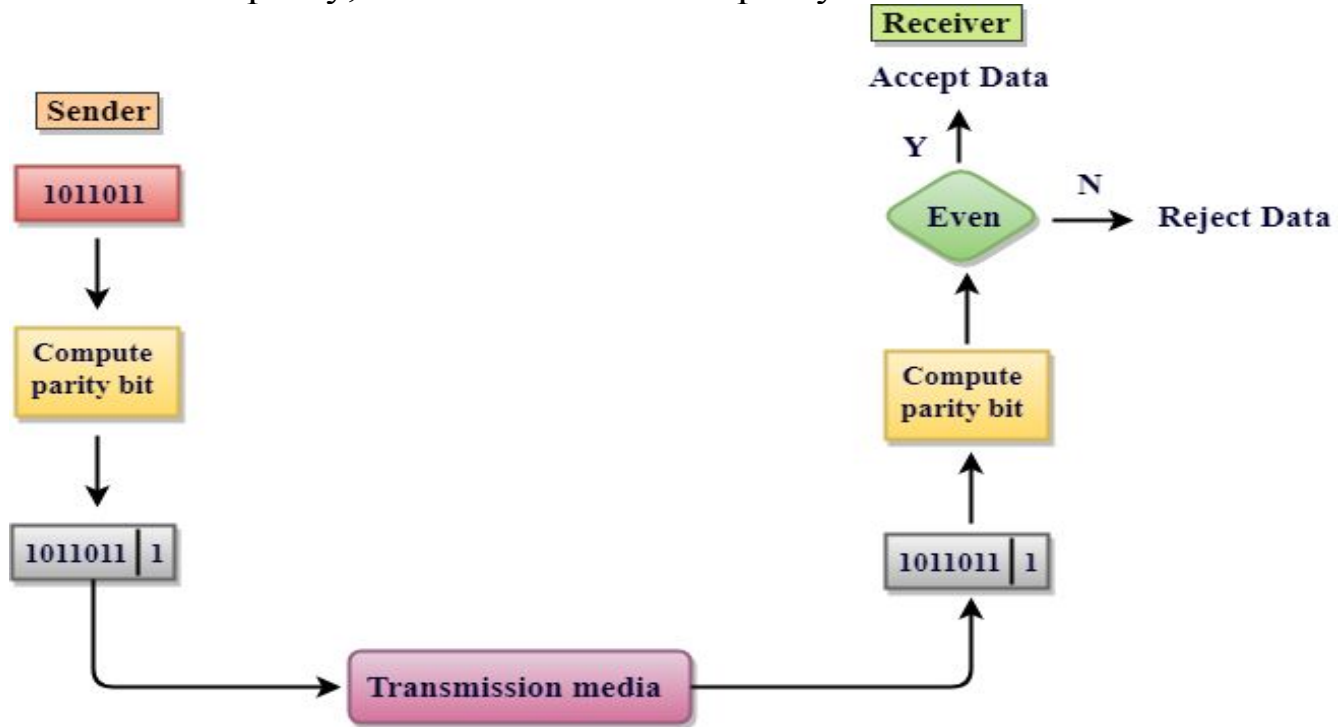
- ❖ Error detection codes
- ❖ Error correction codes

Error Detection Techniques



Parity Check

Parity check is done by adding an extra bit, called parity bit to the data to make number of 1s either even in case of even parity, or odd in case of odd parity.



Parity Check

Parity check is done by adding an extra bit, called parity bit to the data to make number of 1s either even in case of even parity, or odd in case of odd parity.

While creating a frame, the sender counts the number of 1s in it and adds the parity bit in following way

- In case of even parity: If number of 1s is even then parity bit value is 0. If number of 1s is odd then parity bit value is 1.
- In case of odd parity: If number of 1s is odd then parity bit value is 0. If number of 1s is even then parity bit value is 1.

On receiving a frame, the receiver counts the number of 1s in it. In case of even parity check, if the count of 1s is even, the frame is accepted, otherwise it is rejected. Similar rule is adopted for odd parity check.

Parity Check

Example of Even Parity Code

Binary Code	Even Parity bit	Even Parity Code
000	0	0000
001	1	0011
010	1	0101
011	0	0110

Example of Odd Parity Code

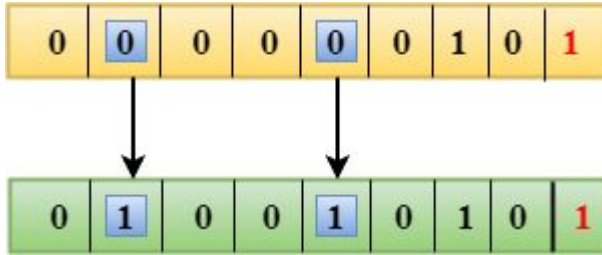
Binary Code	Odd Parity bit	Odd Parity Code
000	1	0001
001	0	0010
010	0	0100
011	1	0111

Parity check is suitable for single bit error detection only.

Drawbacks Of Single Parity Checking

It can only detect single-bit errors which are very rare.

If two bits are interchanged, then it cannot detect the errors.



Performance can be improved by using Two-Dimensional Parity Check which organizes the data in the form of a table.

Checksum

A Checksum is an error detection technique based on the concept of redundancy. The checksum is used in the Internet by several protocols although not at the data link layer.

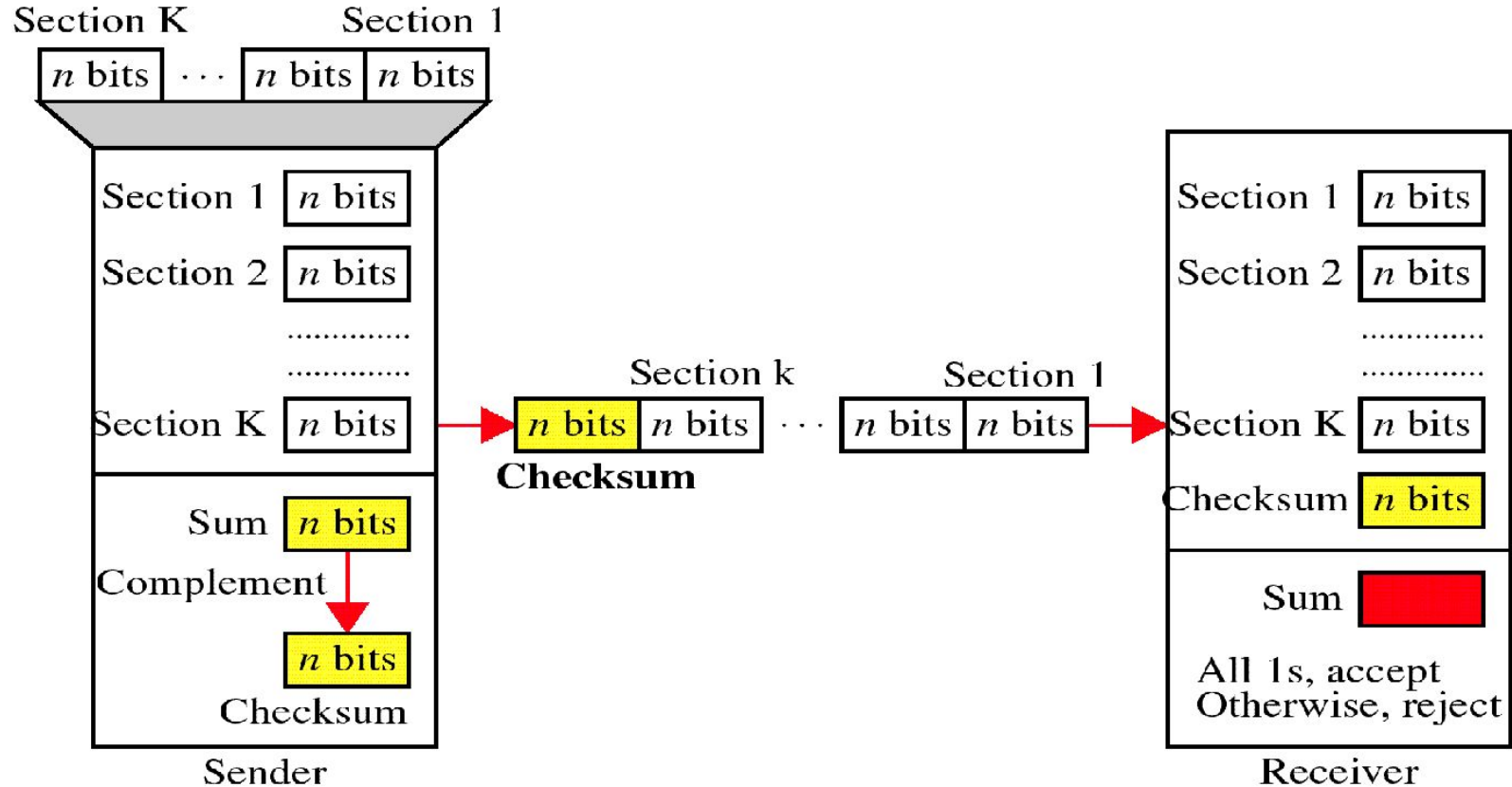
Checksum Sender

1. The Sender follows the given steps:
2. The block unit is divided into k sections, and each of n bits.
3. All the k sections are added together by using one's complement to get the sum.
4. The sum is complemented and it becomes the checksum field.
5. The original data and checksum field are sent across the network.

Checksum Checker

1. The Receiver follows the given steps:
2. The block unit is divided into k sections and each of n bits.
3. All the k sections are added together by using one's complement algorithm to get the sum.
4. The sum is complemented.
5. If the result of the sum is zero, then the data is accepted otherwise the data is discarded.

Checksum



Checksum Example

- Suppose our data is a list of five 4-bit numbers that we want to send to a destination.
- For example, if the set of numbers is (7, 11, 12, 0, 6)
- Sum of the numbers is 36
- The data transmitted (7, 11, 12, 0, 6, -36).
- Receiver will add all the numbers including checksum
- In this case summation = 0 meaning that the data is received correctly
- If the sum is non zero then it is assumed that data has error

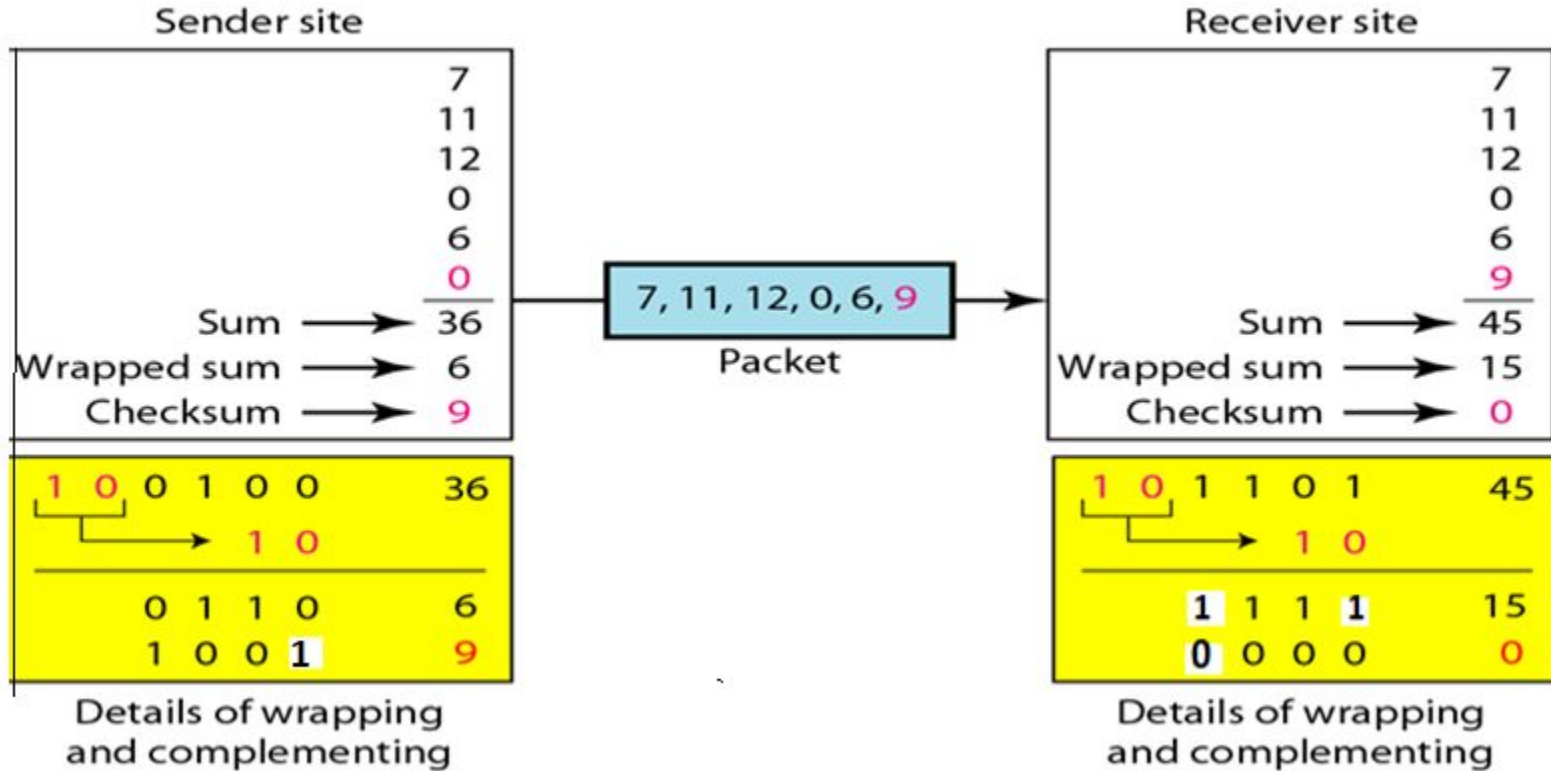
Checksum Example

- Let us redo Exercise using one's complement arithmetic.
- The sender initializes the checksum to 0 and adds all data items and the checksum (the checksum is considered as one data item).
- The result is 36. However, 36 cannot be expressed in 4 bits. **The extra two bits are wrapped and added with the sum to create the wrapped sum value 6.**
- The sum is then complemented, resulting in the checksum value 9 ($15 - 6 = 9$). The sender now sends six data items to the receiver including the checksum 9.

Checksum Example

- The receiver follows the same procedure as the sender.
- It adds all data items (including the checksum); **the result is 45. The sum is wrapped and becomes 15.**
- The wrapped sum is complemented and becomes 0.
- Since the value of the checksum is 0, this means that the data is not corrupted.
- The receiver drops the checksum and keeps the other data items.
- If the checksum is not zero, the entire packet is dropped.

Checksum Example



Checksum

Advantages

- **Simple to implement** – It is very simple to implement and uses very minimal hardware and software resources.
- **Efficient** – Checksum codes can detect the most common errors in digital communication.
- **Fast** – Checksum codes can be computed very quickly because there is nothing much to do here, which is why it is suitable for real-time data packet transmissions.
- **Low Latency** – it won't use many resources and very few computations that's why latency is very low here.

Checksum

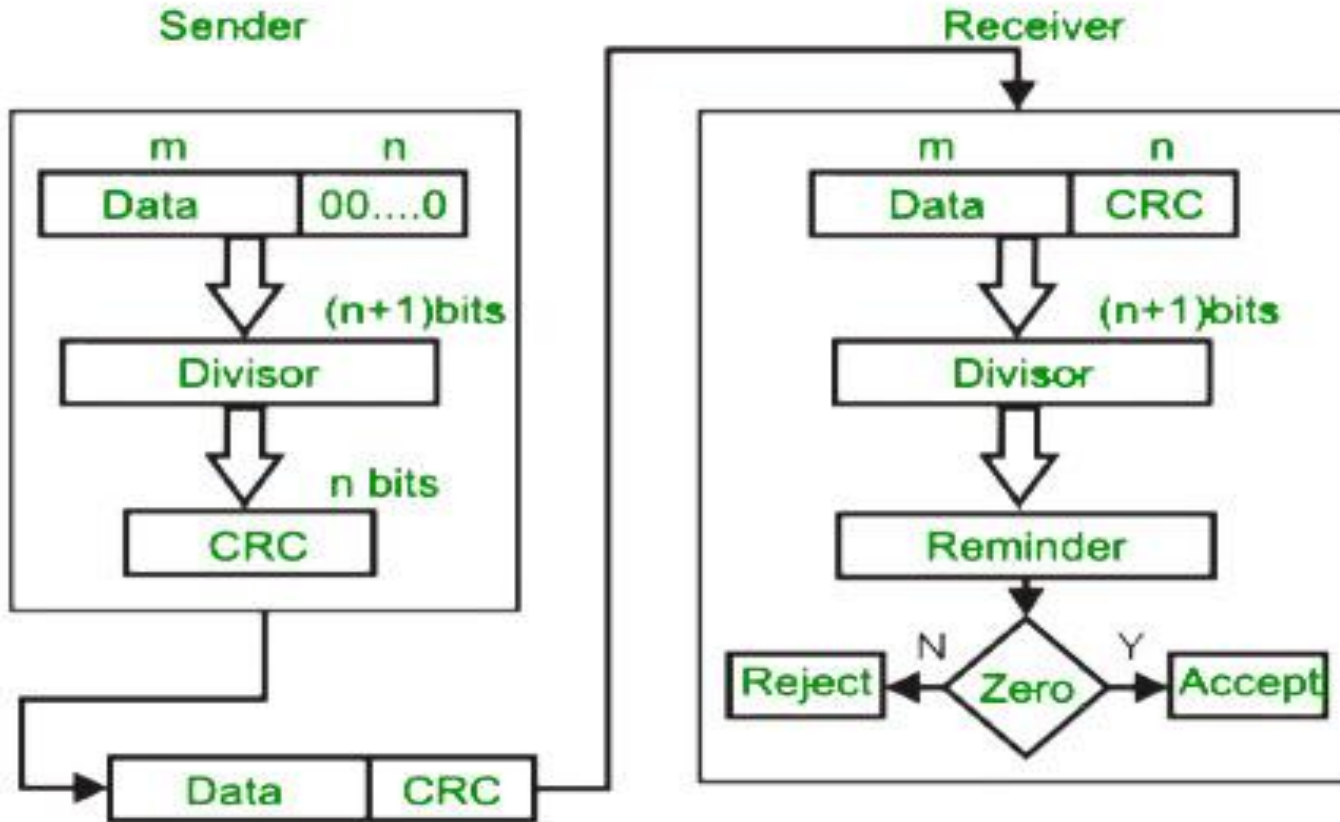
Disadvantages

- **Limited Error Detection Capability** – it is versatile and easy to implement having less complexity but the main problem is it is limited to very common errors in other words it can detect very common errors.
- **Limited Error Correction Capability** – Checksum codes can detect errors in the data packet but they can't correct them. So it is only an error detection technique.
- **Susceptible to errors** – Checksum codes can be susceptible to errors, especially in cases where multiple errors occur in the same data packet.
- **Sensitivity to the Data Packet size** – checksum codes are sensitive to the length of the data packet, if the size of the data packet is too small then very high chances are there that it might miss the error. While the data packet size is large, the overhead of the checksum increases.

Cyclic Redundancy Check (CRC)

- A cyclic redundancy check (CRC) is an error-detecting code commonly used in digital networks and storage devices to detect accidental changes to raw data.
- Unlike the checksum scheme, which is based on addition, CRC is based on binary division.
- In CRC, a sequence of redundant bits, called cyclic redundancy check bits, are appended to the end of the data unit so that the resulting data unit becomes exactly divisible by a second, predetermined binary number.
- At the destination, the incoming data unit is divided by the same number. If at this step there is no remainder, the data unit is assumed to be correct and is therefore accepted.
- A remainder indicates that the data unit has been damaged in transit and therefore must be rejected.

Cyclic Redundancy Check (CRC)



Cyclic Redundancy Check (CRC)

Modulo 2 Division:

The process of modulo-2 binary division is the same as the familiar division process we use for decimal numbers. Just that instead of subtraction, we use XOR here.

- In each step, a copy of the divisor (or data) is XORed with the k bits of the dividend (or key).
- The result of the XOR operation (remainder) is $(n-1)$ bits, which is used for the next step after 1 extra bit is pulled down to make it n bits long.
- When there are no bits left to pull down, we have a result. The $(n-1)$ -bit remainder which is appended at the sender side.

Standard Polynomials

CRC-12

$$x^{12} + x^{11} + x^3 + x + 1$$

CRC-16

$$x^{16} + x^{15} + x^2 + 1$$

CRC-ITU

$$x^{16} + x^{12} + x^5 + 1$$

CRC-32

$$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$$

$$\begin{aligned}\text{Generator polynomial} &= x^3 + x^2 + 1 \\ &= 1x^3 + 1x^2 + 0x^1 + 1x^0\end{aligned}$$

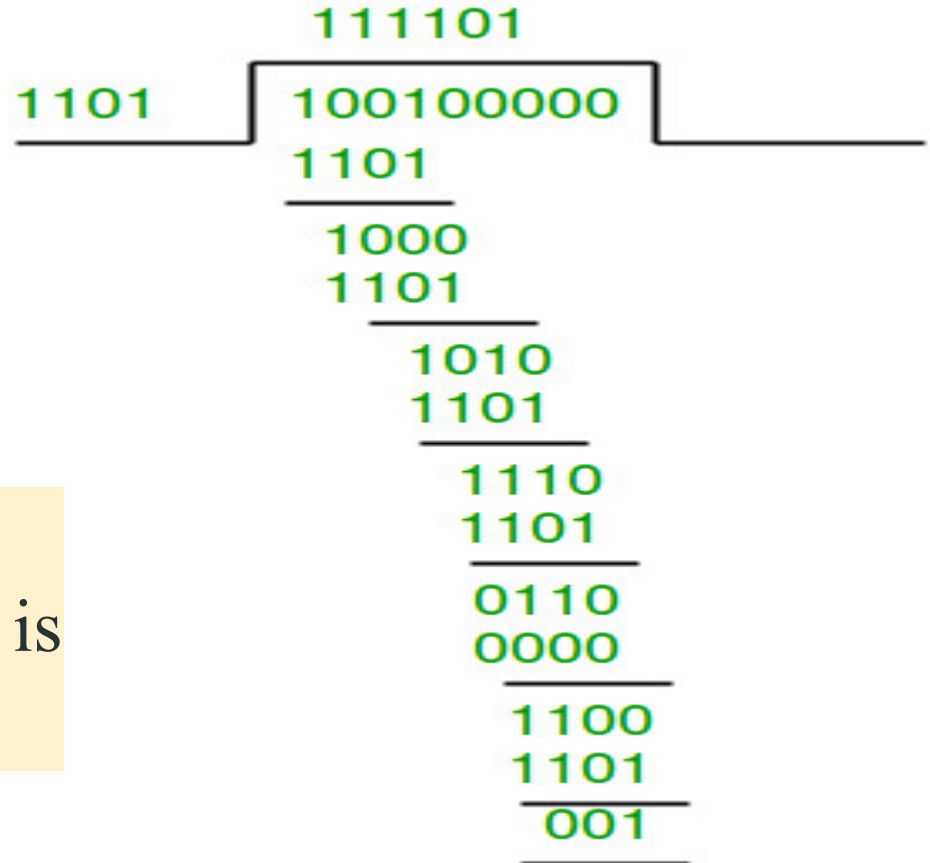
$$\text{Key} = 1101$$

CRC: Example 1- Sender side

Data word to be sent – 100100

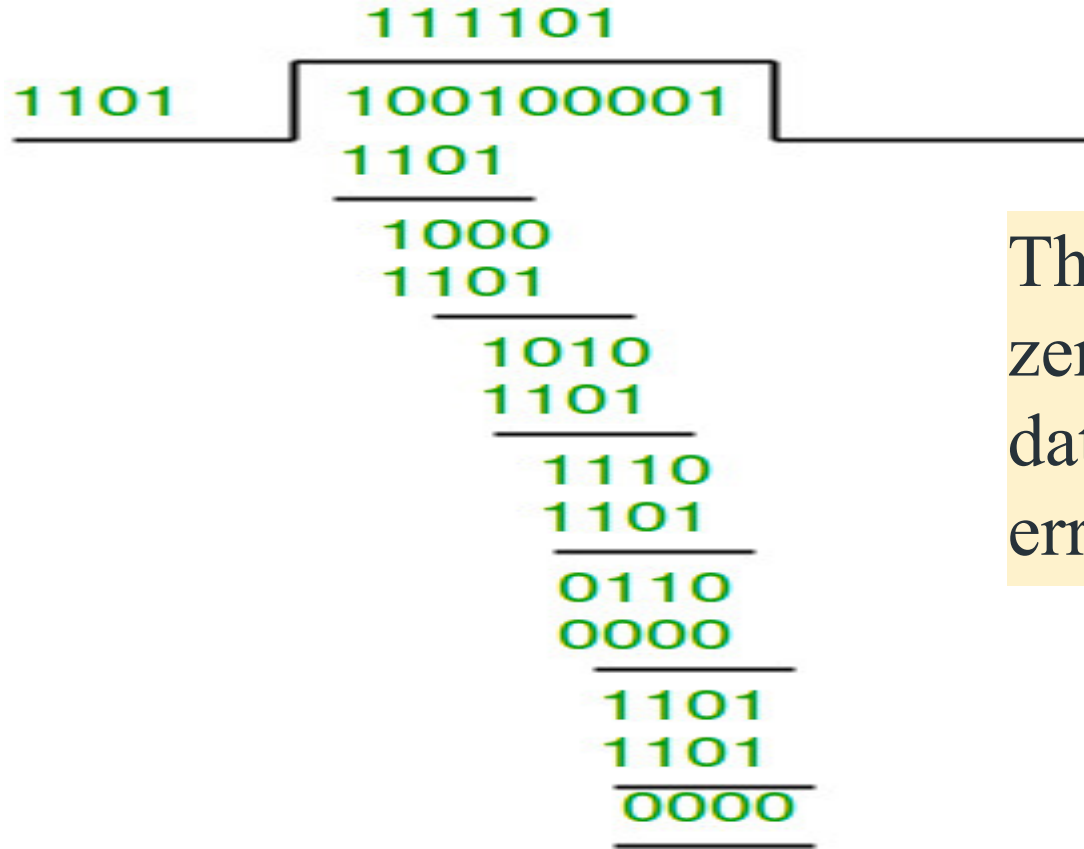
Key = 1101

The remainder is 001 and
hence the encoded data sent is
100100001.



CRC: Example 1- Receiver side

Code word received at the receiver side is 100100001

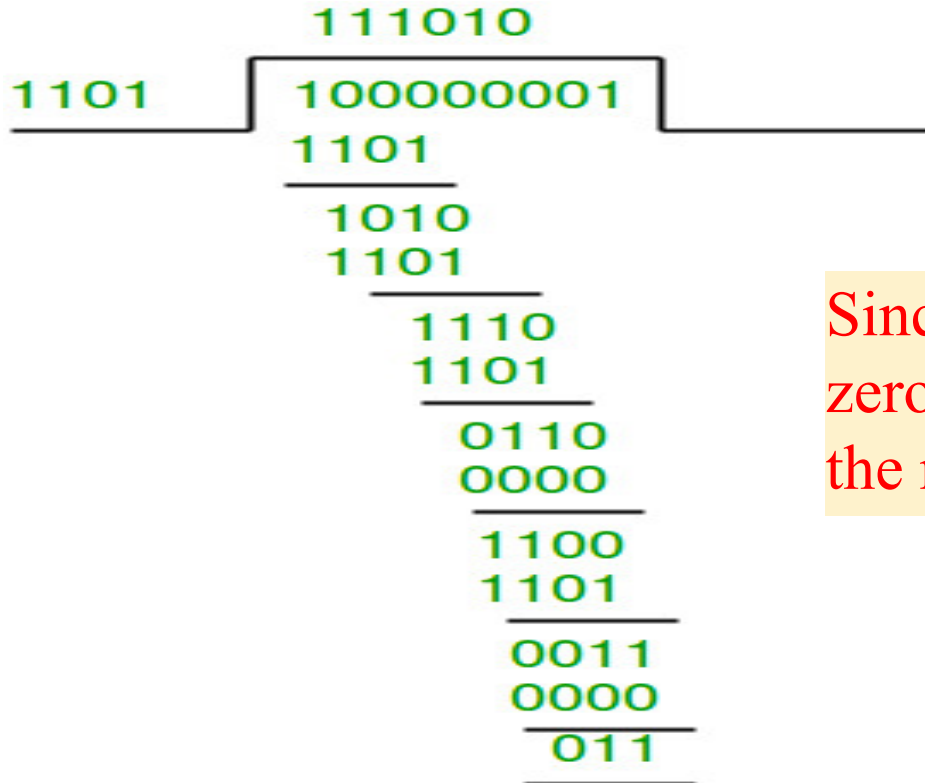


The remainder is all zeros. Hence, the data received has no error.

CRC: Example 2- Receiver side

Code word received-100000001

Key - 1101

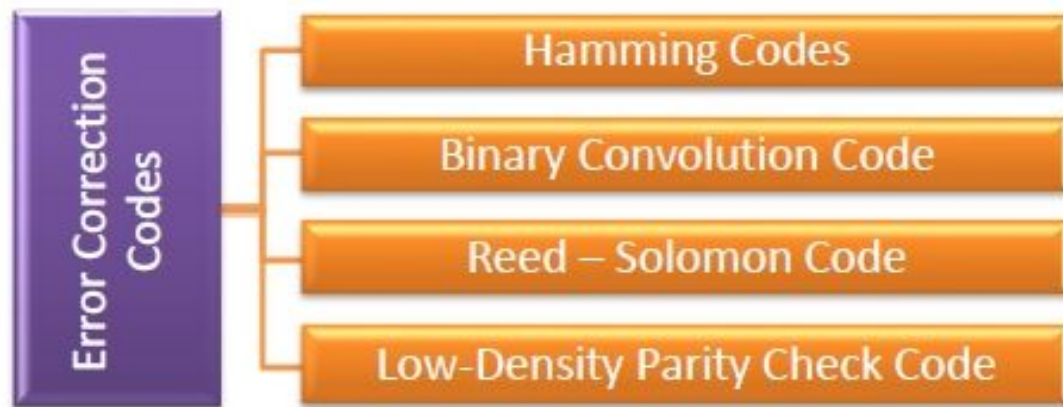


Since the remainder is not all zeroes, the error is detected at the receiver side

Error-correcting codes

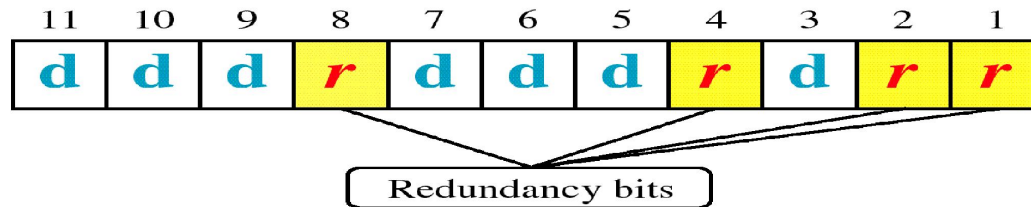
- Error-correcting codes (ECC) are a sequence of numbers generated by specific algorithms for detecting and removing errors in data that has been transmitted over noisy channels.
- Error correcting codes ascertain the exact number of bits that has been corrupted and the location of the corrupted bits

Types of Error Correcting Codes



Hamming Code

- Hamming code is a block code that is capable of detecting up to two simultaneous bit errors and correcting single-bit errors
- The source encodes the message by inserting redundant bits within the message.
- These redundant bits are extra bits that are generated and inserted at specific positions in the message itself to enable error detection and correction.
- When the destination receives this message, it performs recalculations to detect errors and find the bit position that has error.



Hamming Code: Encoding

Encoding a message by Hamming Code

Step 1 – Calculation of the number of redundant bits.

Step 2 – Positioning the redundant bits.

Step 3 – Calculating the values of each redundant bit.

Step 1

If the message contains m number of data bits, then r number of redundant bits are added to it.

The value of r must satisfy the following relation: $2^r \geq m+r+1$.

For example: If data bits $m = 7$ then $r = 4$

Length of the code-word = $7 + 4 = 11$

Hamming Code: Encoding

Step 2

The r redundant bits placed at bit positions of powers of 2, i.e. 1, 2, 4, 8, 16 etc.

They are referred as r_1 (at position 1), r_2 (at position 2), r_3 (at position 4), r_4 (at position 8) and so on.

Step 3

The redundant bits are parity bits. A parity bit is an extra bit that makes the number of 1s either even or odd. The two types of parity are –

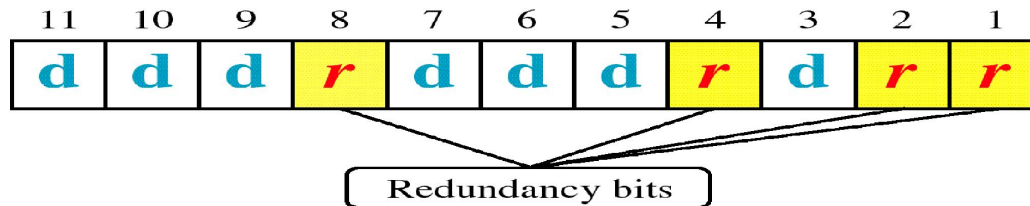
Even Parity – Here the total number of bits in the message is made even.

Odd Parity – Here the total number of bits in the message is made odd.

Hamming Code: Encoding

Each redundant bit, r_i , is calculated as the parity, generally even parity, based upon its bit position. It covers all bit positions whose binary representation includes a 1 in the i^{th} position except the position of r_i

- r_1 is the parity bit for all data bits in positions whose binary representation includes a 1 in the least significant position excluding 1 (3, 5, 7, 9, 11 and so on)
- r_2 is the parity bit for all data bits in positions whose binary representation includes a 1 in the position 2 from right except 2 (3, 6, 7, 10, 11 and so on)
- r_3 is the parity bit for all data bits in positions whose binary representation includes a 1 in the position 3 from right except 4 (5-7, 12-15, 20-23 and so on)



Hamming Code: Encoding

Position	R8	R4	R2	R1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1

Sometimes the redundant bits are named as per their position eg. R1, R2, R4, R8 etc. instead of R1, R2, R3, R4. In both the cases calculation process remains the same

R1 -> 1,3,5,7,9,11

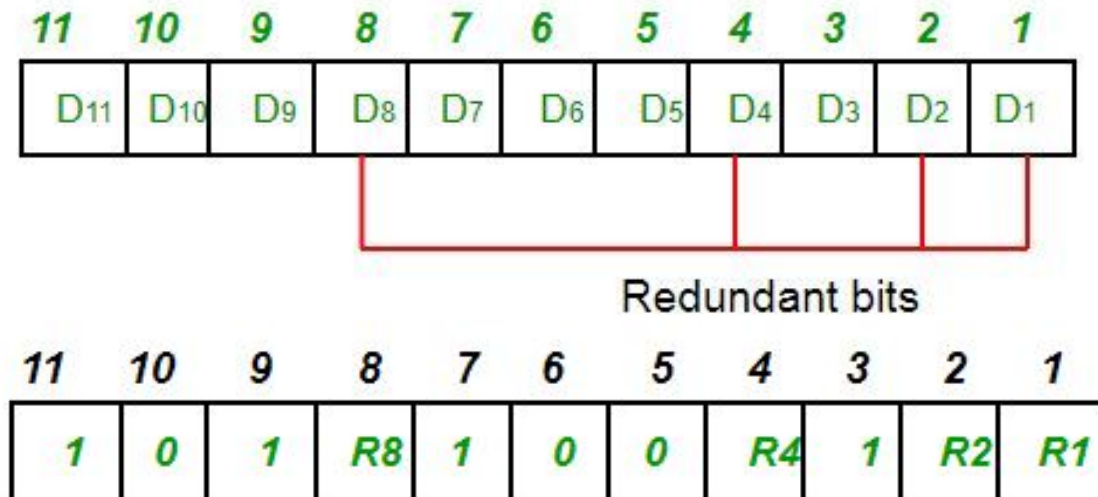
R2 -> 2,3,6,7,10,11

R4 -> 4,5,6,7

R8 -> 8,9,10,11

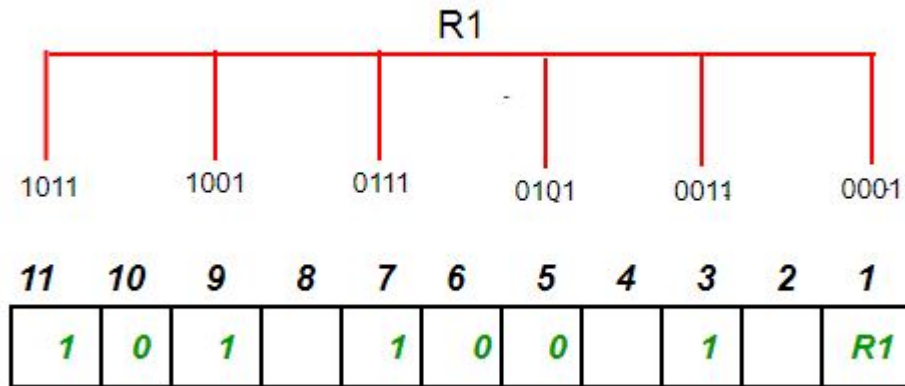
Hamming Code: Encoding

- The number of data bits = 7 (**m**)
- The number of redundant bits = 4 (**r**)
- The total number of bits = 11 (**n = m + r**)
- The redundant bits are placed at positions corresponding to power of 2- 1, 2, 4, 8
- data to be transmitted is 1011001



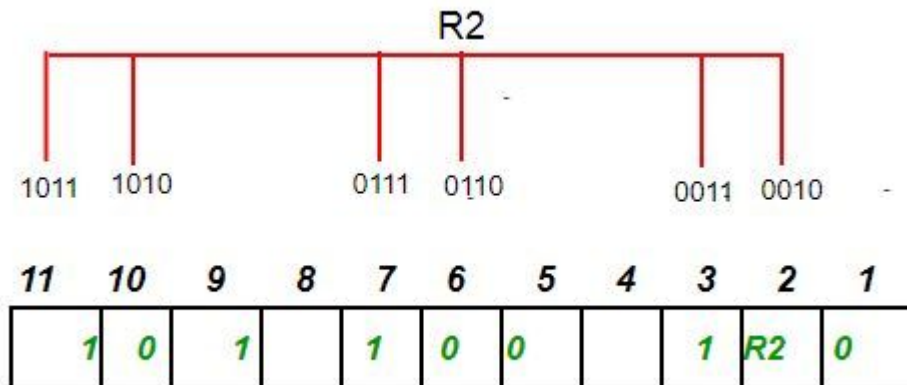
Hamming Code: Encoding

Determining the Parity bits:



To find the redundant bit R1, we check for even parity. Since the total number of 1's in all the bit positions corresponding to R1 is an even number.

R1 = 0

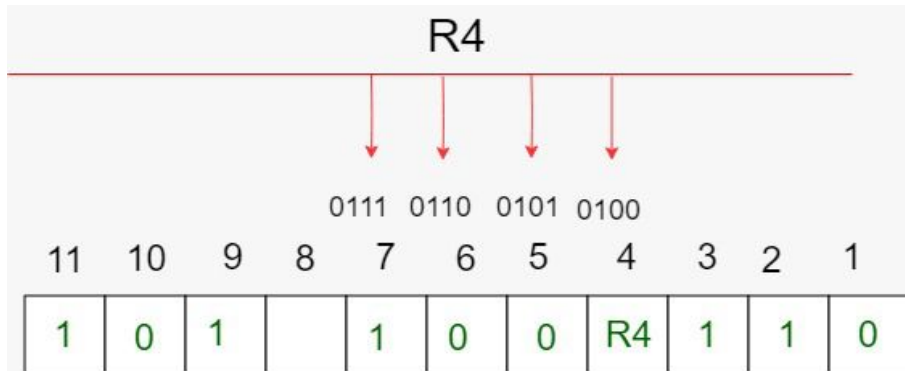


To find the redundant bit R2, we check for even parity. Since the total number of 1's in all the bit positions corresponding to R2 is odd.

R2 = 1

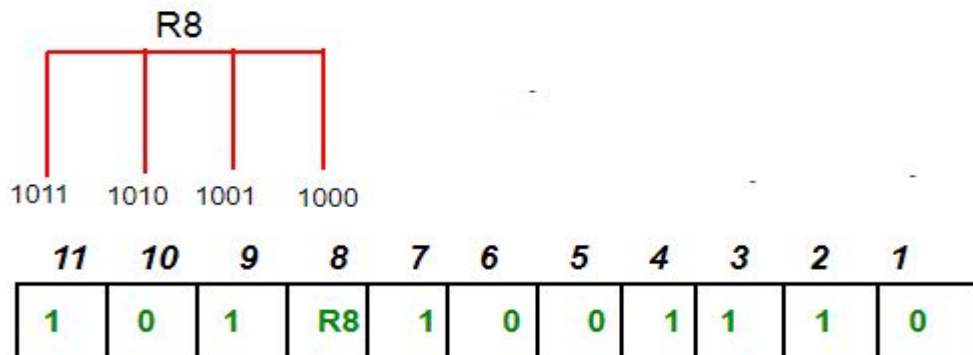
Hamming Code: Encoding

Determining the Parity bits:



To find the redundant bit R4, we check for even parity. Since the total number of 1's in all the bit positions corresponding to R4 is odd

$$R4 = 1$$



To find the redundant bit R8, we check for even parity. Since the total number of 1's in all the bit positions corresponding to R8 is an even number

$$R8 = 0$$

Hamming Code: Encoding

Thus, the data transferred is:

11	10	9	8	7	6	5	4	3	2	1
1	0	1	0	1	0	0	1	1	1	0

Hamming Code: Encoding

Consider an Example, **Data: 1001101** - Use modulo - 2 to calculate parity bits

Data: 1 0 0 1 1 0 1



Data	1	0	0		1	1	0		1		
------	---	---	---	--	---	---	---	--	---	--	--

Adding r_1	1	0	0		1	1	0		1		1
--------------	---	---	---	--	---	---	---	--	---	--	---

Adding r_2	1	0	0		1	1	0		1	0	1
--------------	---	---	---	--	---	---	---	--	---	---	---

Adding r_4	1	0	0		1	1	0	0	1	0	1
--------------	---	---	---	--	---	---	---	---	---	---	---

Adding r_8	1	0	0	1	1	1	0	0	1	0	1
--------------	---	---	---	---	---	---	---	---	---	---	---



Code: 1 0 0 1 1 1 0 0 1 0 1

$$R_1 = D_3, D_5, D_7, D_9, D_{11} \Rightarrow 1+0+1+0+1 \Rightarrow 1.$$

$$R_2 = D_3, D_6, D_7, D_{10}, D_{11} \Rightarrow 1+1+1+0+1 \Rightarrow 0.$$

$$R_4 = D_5, D_6, D_7 \Rightarrow 0+1+1 \Rightarrow 0.$$

$$R_8 = D_9, D_{10}, D_{11} \Rightarrow 0+0+1 \Rightarrow 1.$$

Hamming Code: Encoding

Consider another Example where we send data stream **1000001** [ASCII 'A']

Now we will encode the message

Use modulo - 2 to calculate parity bits

$$R1 = D3, D5, D7, D9, D11 \Rightarrow 1+0+0+0+1 \Rightarrow 0.$$

$$R2 = D3, D6, D7, D10, D11 \Rightarrow 1+0+0+0+1 \Rightarrow 0.$$

$$R4 = D5, D6, D7 \Rightarrow 0+0+0 \Rightarrow 0.$$

$$R8 = D9, D10, D11 \Rightarrow 0+0+1 \Rightarrow 1.$$

Data stream to be sent on transmission line from sender

1	0	0	1	0	0	0	0	1	0	0
11	10	9	8	7	6	5	4	3	2	1

Hamming Code: Decoding

Once the receiver gets an incoming message, it performs recalculations to detect errors and correct them. The steps for recalculation are –

- **Step 1** – Calculation of the number of redundant bits.
- **Step 2** – Positioning the redundant bits.
- **Step 3** – Parity checking.
- **Step 4** – Error detection and correction

Step 1

• Using the same formula as in encoding, the number of redundant bits are ascertained.

Step 2

• The r redundant bits placed at bit positions of powers of 2, i.e. 1, 2, 4, 8, 16 etc.

Hamming Code: Decoding

Step 3

Parity bits are calculated based upon the data bits and the redundant bits using the same rule as during generation of c_1, c_2, c_3, c_4 etc. Thus

$c_1 = \text{parity}(1, 3, 5, 7, 9, 11 \text{ and so on})$

$c_2 = \text{parity}(2, 3, 6, 7, 10, 11 \text{ and so on})$

$c_4 = \text{parity}(4-7, 12-15, 20-23 \text{ and so on})$

Step 4

The decimal equivalent of the parity bits binary values is calculated. If it is 0, there is no error. Otherwise, the decimal value gives the bit position which has error.

For example, if $c_4c_3c_2c_1 = 1001$, it implies that the data bit at position 9

The bit is flipped to get the correct message.

Hamming Code: Example

- when this stream is received at receiver same process will be followed for decoding if we are getting same values for parity bits as we got at sender side then we say that transmission was error free
- Consider an erroneous stream where 5th bit is flipped

1	0	0	1	0	0	1	0	1	0	0
11	10	9	8	7	6	5	4	3	2	1

Data stream received on
transmission line from
sender

Now getting corrupted bit position

$= (C8, CR4, C2, C1)$

$= (0, 1, 0, 1)_2$

$= (5)_{10}$

$C1 = R1, D3, D5, D7, D9, D11 \Rightarrow 0+1+1+0+0+1 \Rightarrow 1.$

$C2 = R2, D3, D6, D7, D10, D11 \Rightarrow 0+1+0+0+0+1 \Rightarrow 0.$

$C4 = R4, D5, D6, D7 \Rightarrow 0+1+0+0 \Rightarrow 1.$

$C8 = R8, D9, D10, D11 \Rightarrow 1+0+0+1 \Rightarrow 0.$

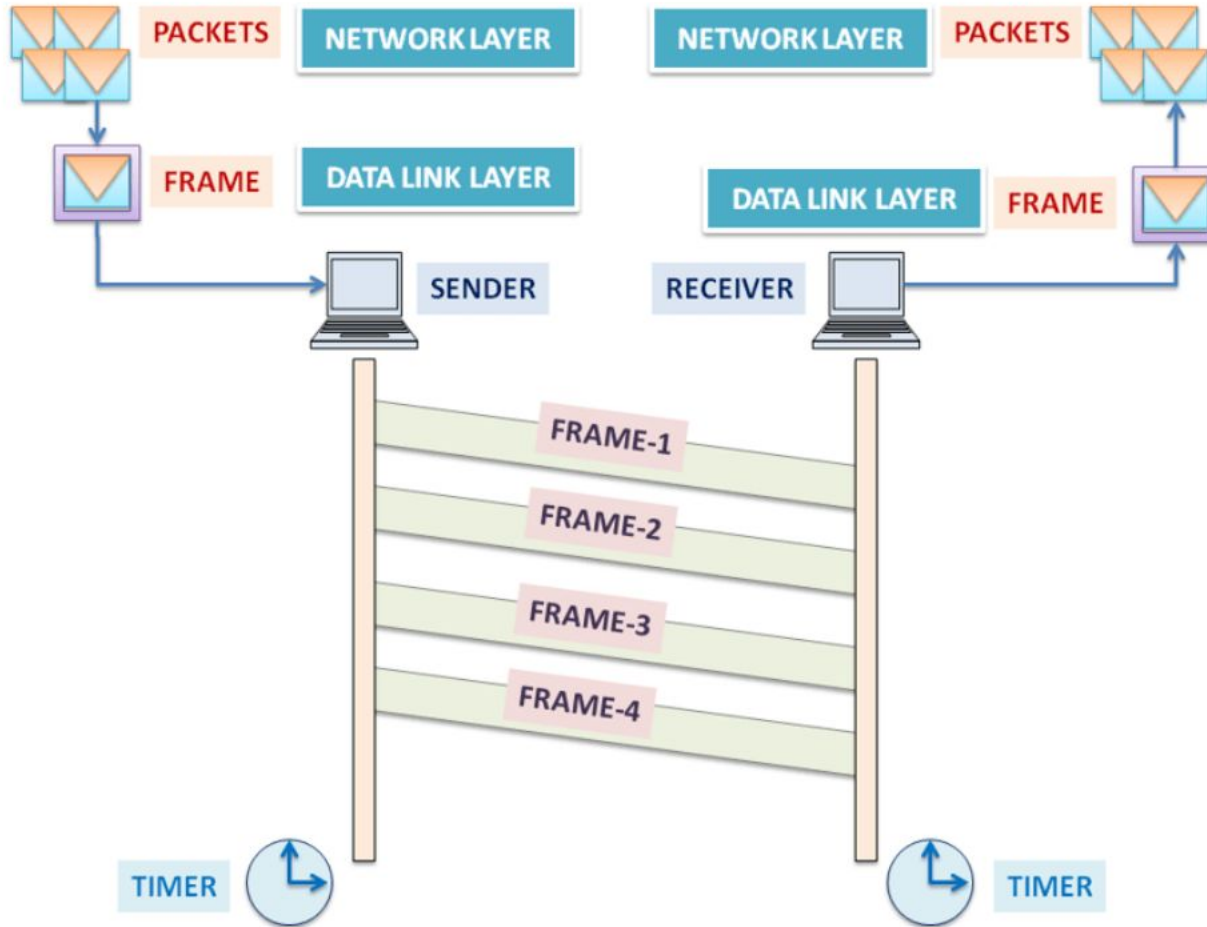
Flow Control and Error control

- The most important responsibilities of the data link layer are **flow control** and **error control**. Collectively, these functions are known as **data link control**.
- **Flow control** refers to a set of procedures used to restrict the amount of data that the sender can send before waiting for acknowledgment.
- **Error control** in the data link layer is based on automatic repeat request, which is the retransmission of data.

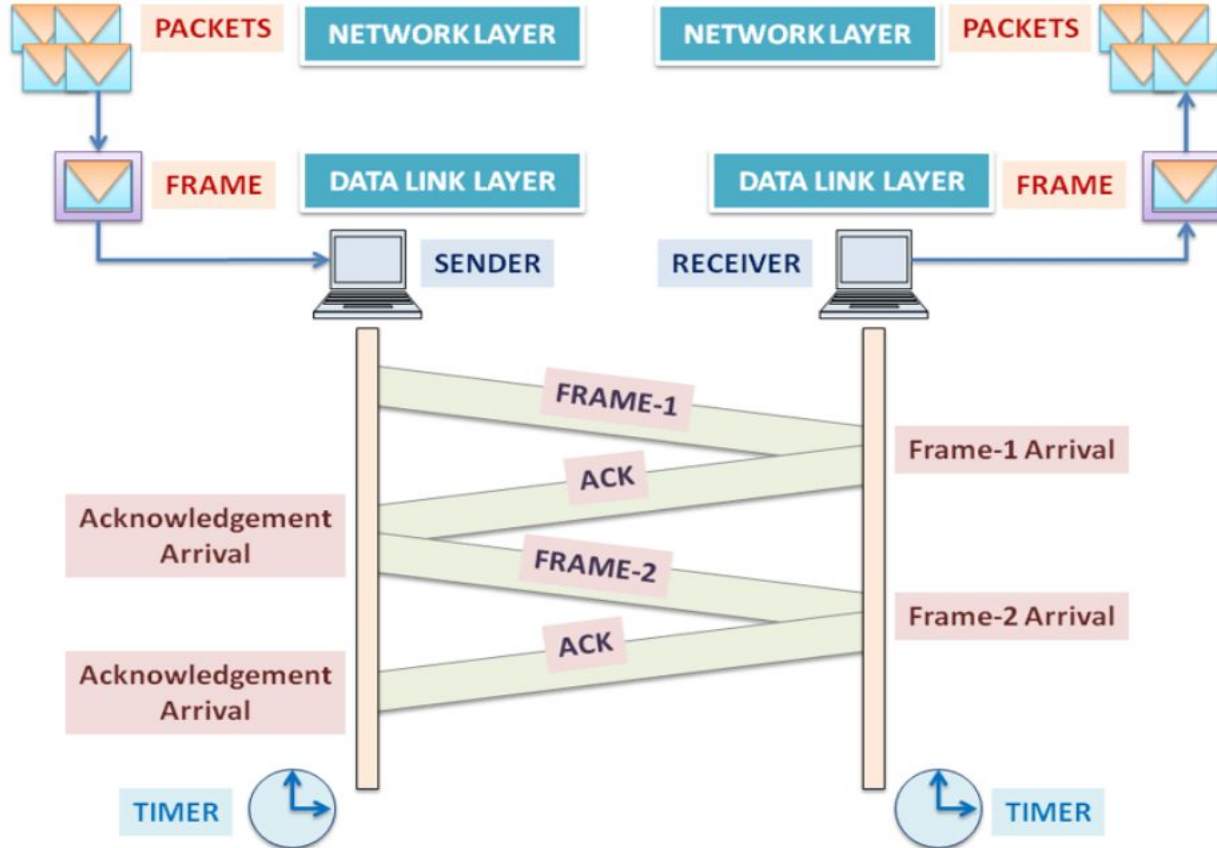
Elementary data link control Protocols

1. Unrestricted Simplex protocol
2. Simplex stop and wait protocol
3. Simplex protocol for Noisy channels

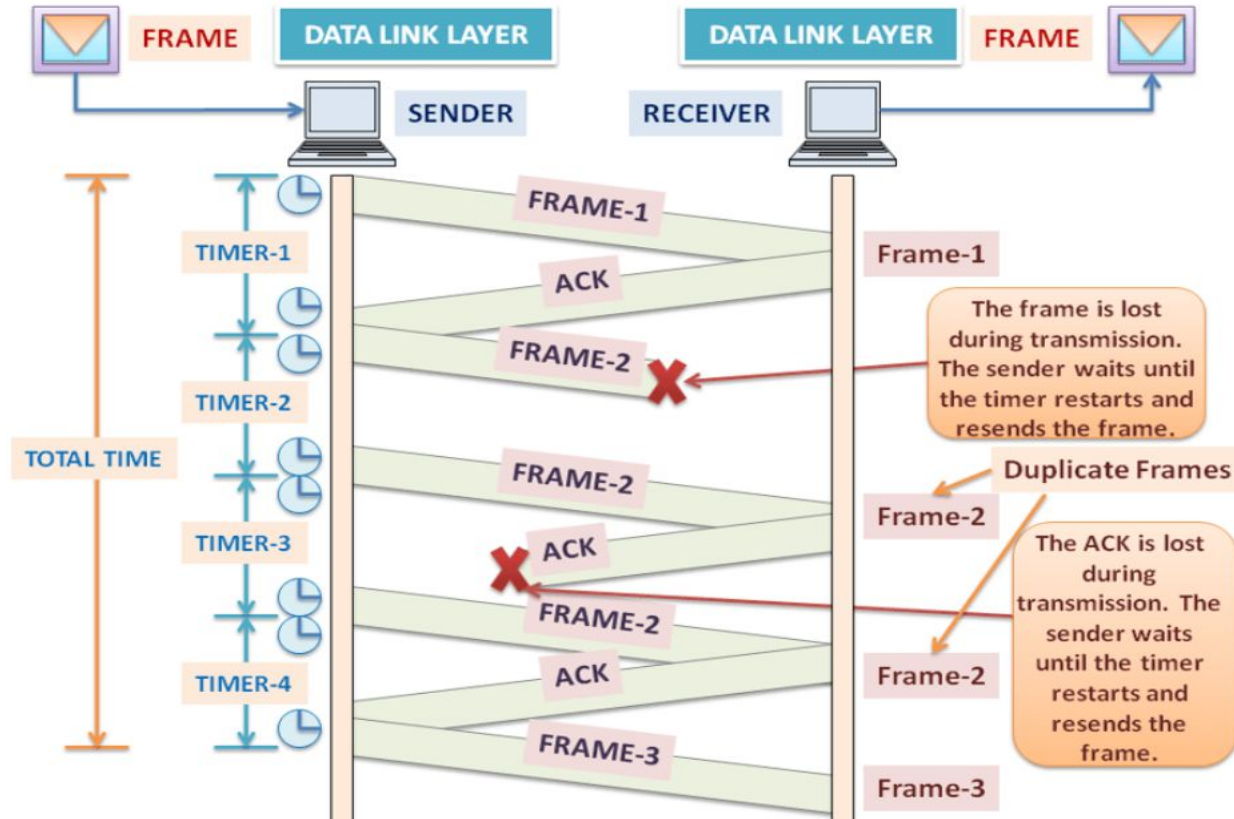
1. Unrestricted Simplex protocol



2. Simplex stop and wait protocol



3. Simplex protocol for Noisy channels



Flow Control

- Flow control is a technique that allows two stations working at different speeds to communicate with each other.
- In data link layer, flow control restricts the number of frames the sender can send before it waits for an acknowledgment from the receiver.
- If the **rate of produced frames is higher than the rate of consumed frames**, frames at the receiving end **need to be buffered** while waiting to be consumed (processed).
- As an unlimited buffer size at the receiving side is not possible, any one of the following steps are followed
 - **Receiving data-link layer drop the frames if its buffer is full.**
 - **To let the receiving data-link layer send a feedback to the sending data-link layer to ask it to stop or slow down.**
- Different data-link-layer protocols use different strategies for flow control

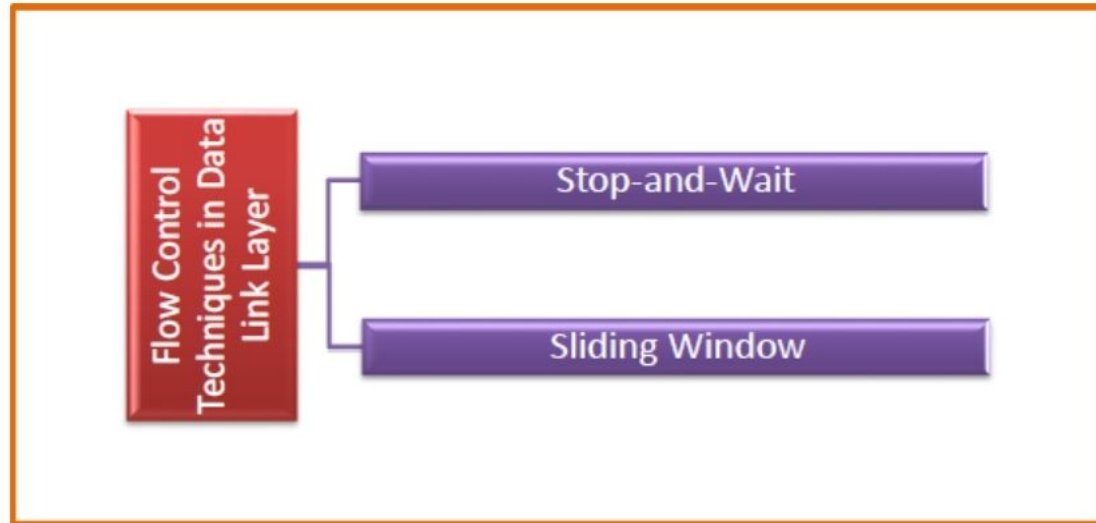
Flow Control can be broadly categorized into two approaches:



• **Feedback based Flow Control** In this method, the sender transmits data and waits for feedback (acknowledgment) from the receiver before sending more data. This ensures that the receiver is ready to process the incoming data. It is commonly used in the data link layer.

• **Rate based Flow Control** This method uses built-in mechanisms in protocols to limit the data transmission rate without requiring acknowledgment from the receiver. It is typically employed in the network and transport layers.

Data link layer uses feedback based flow control mechanisms. There are two main techniques



Stop and Wait

This protocol involves the following transitions

- The sender sends a frame and waits for acknowledgment.
- Once the receiver receives the frame, it sends an acknowledgment frame back to the sender.
- On receiving the acknowledgment frame, the sender understands that the receiver is ready to accept the next frame. So it sends the next frame in queue.

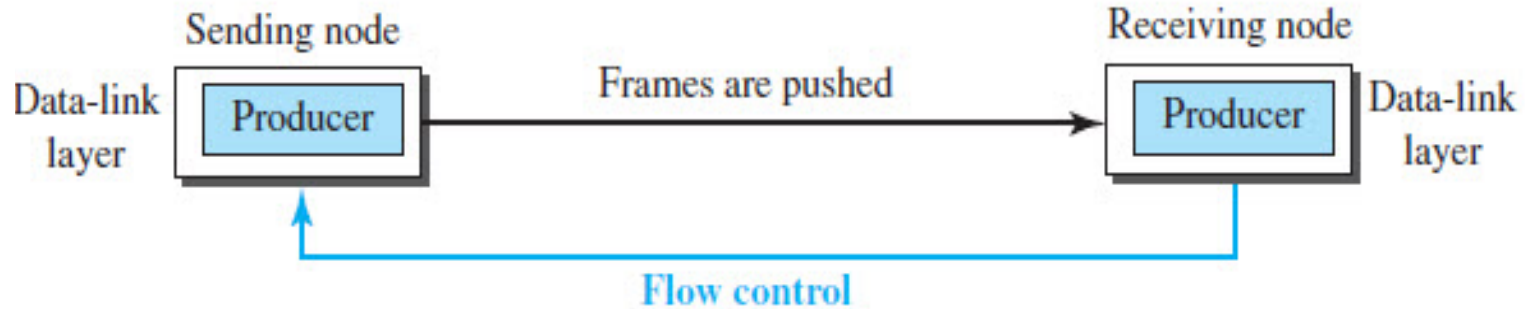
Sliding Window

This protocol improves the efficiency of stop and wait protocol by allowing multiple frames to be transmitted before receiving an acknowledgment.

The working principle of this protocol can be described as follows

- Both the sender and the receiver has finite sized buffers called windows. The sender and the receiver agrees upon the number of frames to be sent based upon the buffer size.
- The sender sends multiple frames in a sequence, without waiting for acknowledgment. When its sending window is filled, it waits for acknowledgment. On receiving acknowledgment, it advances the window and transmits the next frames, according to the number of acknowledgments received.

Flow Control



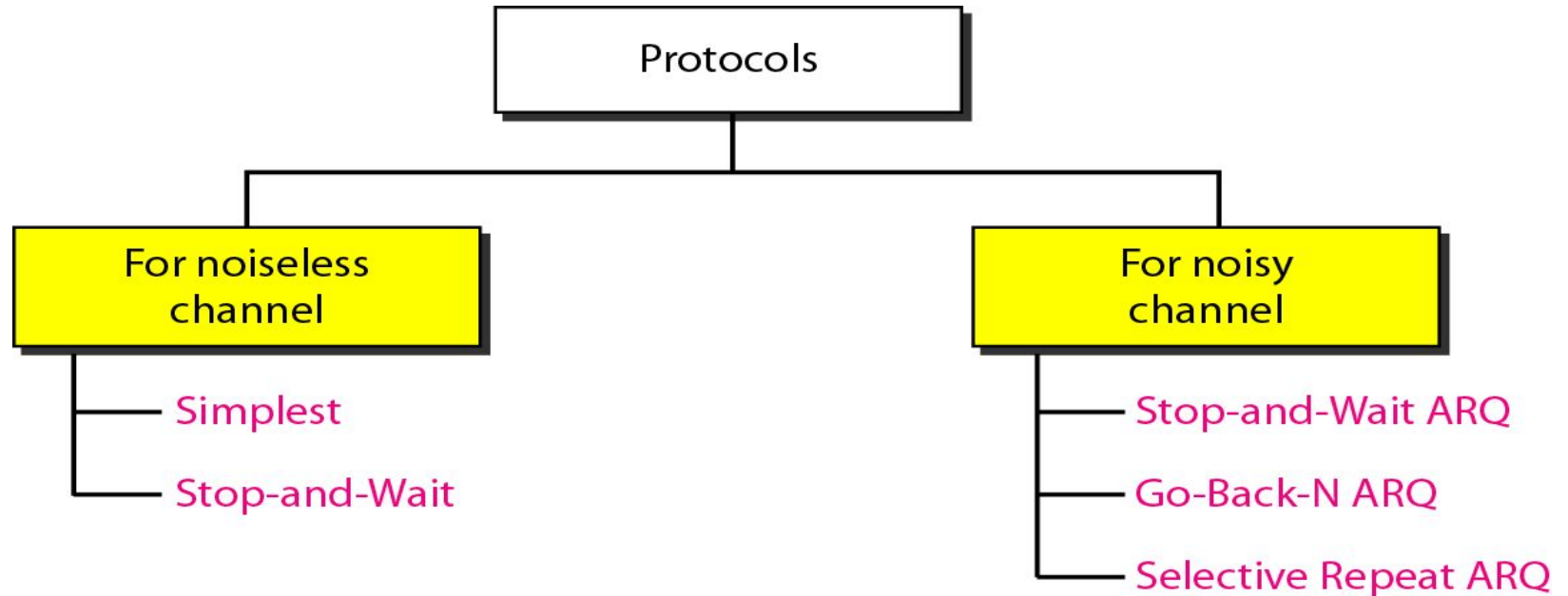
- data-link layer at the sending node tries to push frames toward the data-link layer at the receiving node.
- If the receiving node cannot process and deliver the packet to its network at the same rate that the frames arrive, it becomes overwhelmed with frames.
- Flow control in this case can be feedback from the receiving node to the sending node to stop or slow down pushing frames.

Flow Control

- **Flow and error control can be combined.**
- In a simple situation, the acknowledgment that is sent for flow control can also be used for error control to tell the sender the packet has arrived uncorrupted.
- The lack of acknowledgment means that there is a problem in the sent frame.
- A frame that carries an acknowledgment is normally called an ACK to distinguish it from the data frame.

Taxonomy of protocols

Data link layer protocols are divided into two categories based on whether the transmission channel is noiseless or noisy.



Noiseless Channels

There are two noiseless channels which are as follows

- Simplex channel
- Stop & wait channel

An ideal channel where no frames are lost, duplicated, or corrupted. We introduce two protocols for this type of channel. These two protocols are as follows

- Protocol that does not use flow control: Simplest Protocol
- Protocol that uses the flow control: Stop-and-Wait Protocol

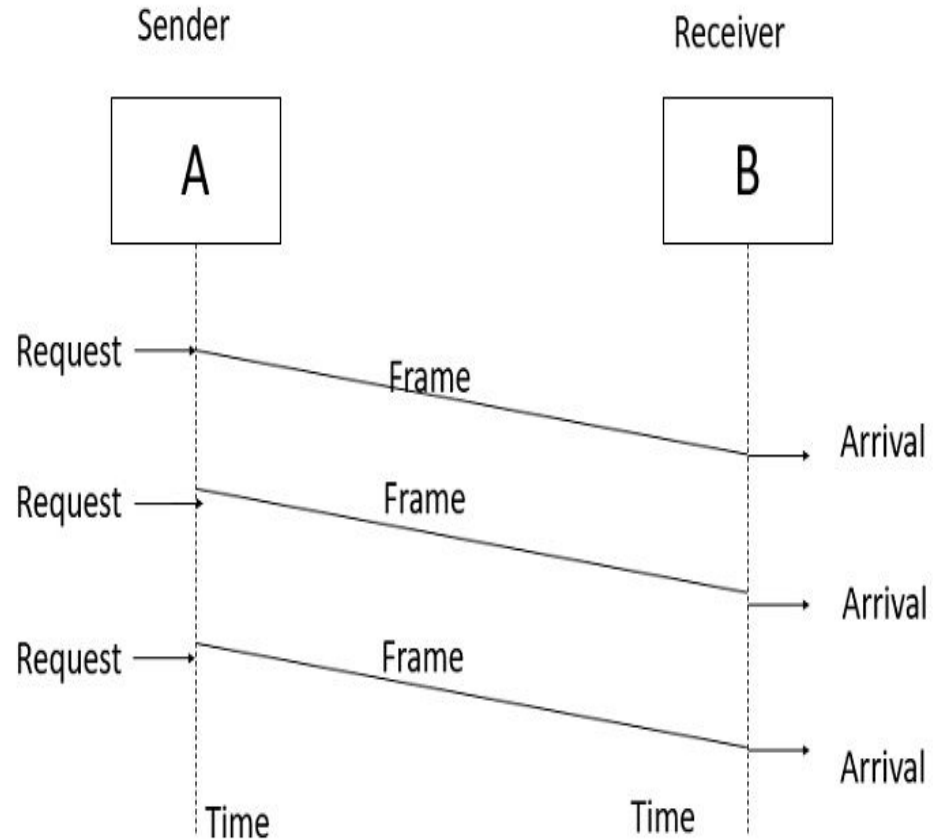
Simplest Protocol

Step 1 – Simplest protocol that does not have flow or error control.

Step 2 – It is a unidirectional protocol where data frames are traveling in one direction that is from the sender to receiver.

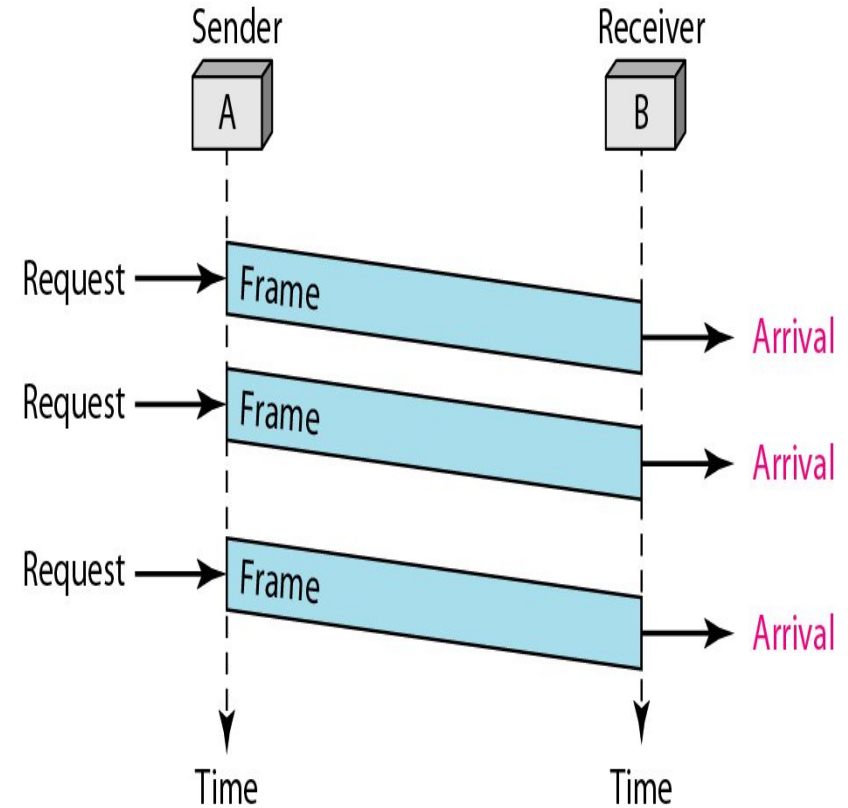
Step 3 – It assumes that

- The receiver can handle any frame it receives with a processing time that is small enough to be negligible
- The data link layer of the receiver immediately removes the header from the frame and hands the data packet to its network layer
- The network layer can also accept the packet immediately



Simple Protocol

- The sender sends a sequence of frames without even thinking about the receiver.
- To send three frames, three events occur at the sender side and three events at the receiver side.
- The data frames are shown by tilted boxes; the height of the box defines the transmission time difference between the first bit and the last bit in the frame.



Stop & Wait Protocol

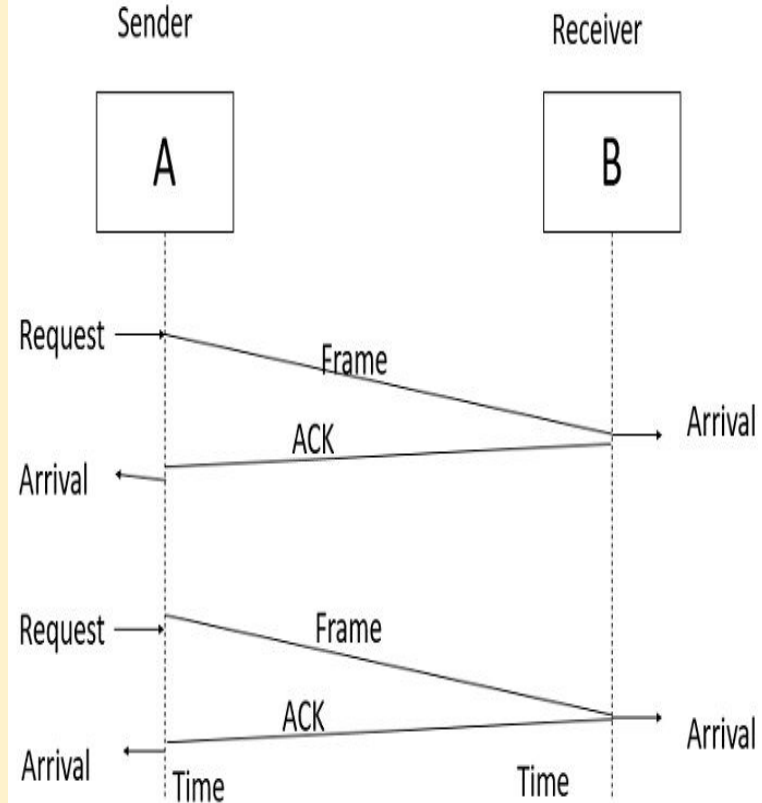
Step 1 – If the data frames arrive faster than they can be processed, the frames must be stored

Step 2 – If it is receiving data from many sources, then it does not have enough storage space

Step 3 – To prevent the receiver from becoming overwhelmed with frames, the sender must slow down. There must be ACK from the receiver to the sender.

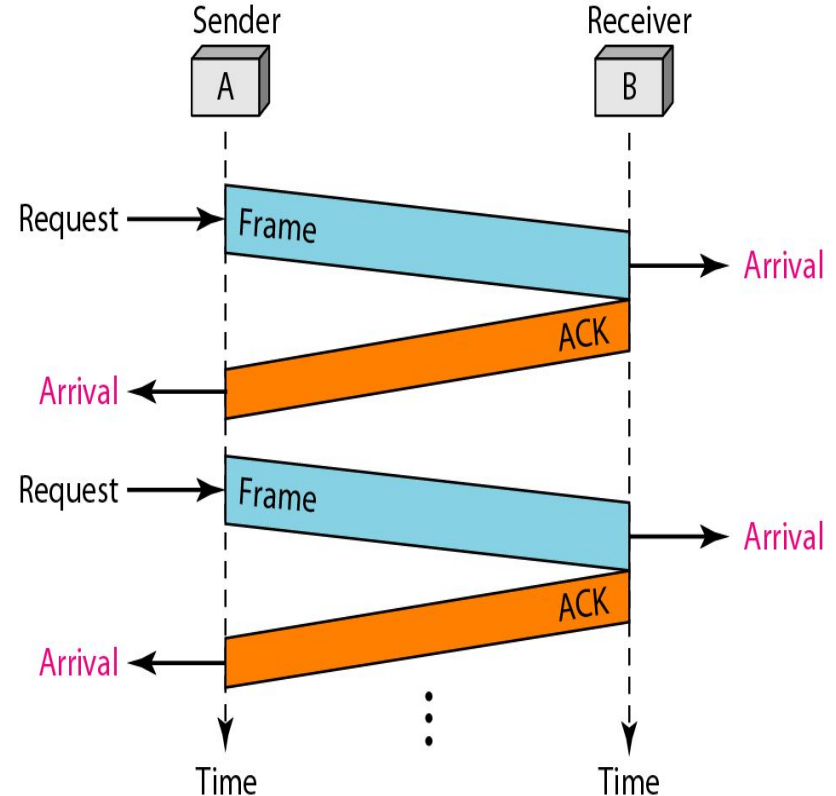
Step 4 – In this protocol the sender sends one frame, stops until it receives confirmation from the receiver, and then sends the next frame.

Step 5 – We still have unidirectional communication for data frames, but auxiliary ACK frames travel from the other direction. We add flow control to the previous protocol.



Stop & Wait Protocol

- The sender sends one frame and waits for feedback from the receiver.
- When the ACK arrives, the sender sends the next frame.
- Note that sending two frames in the protocol involves the sender in four events and the receiver in two events.



Noisy Channel

Noiseless channels are nonexistent.

For noisy channels, following protocols are used for flow and error control

- Stop-and-Wait Automatic Repeat Request
- Go-Back-N Automatic Repeat Request
- Selective Repeat Automatic Repeat Request

Error Control Techniques

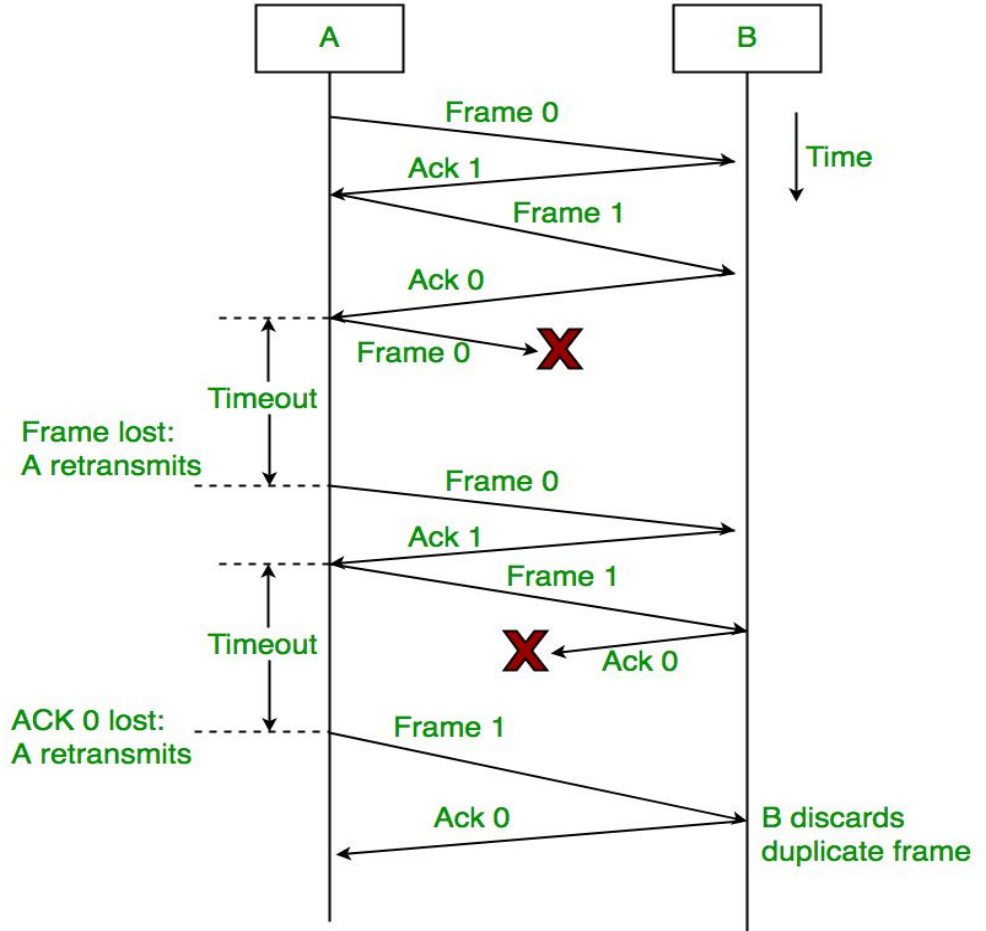
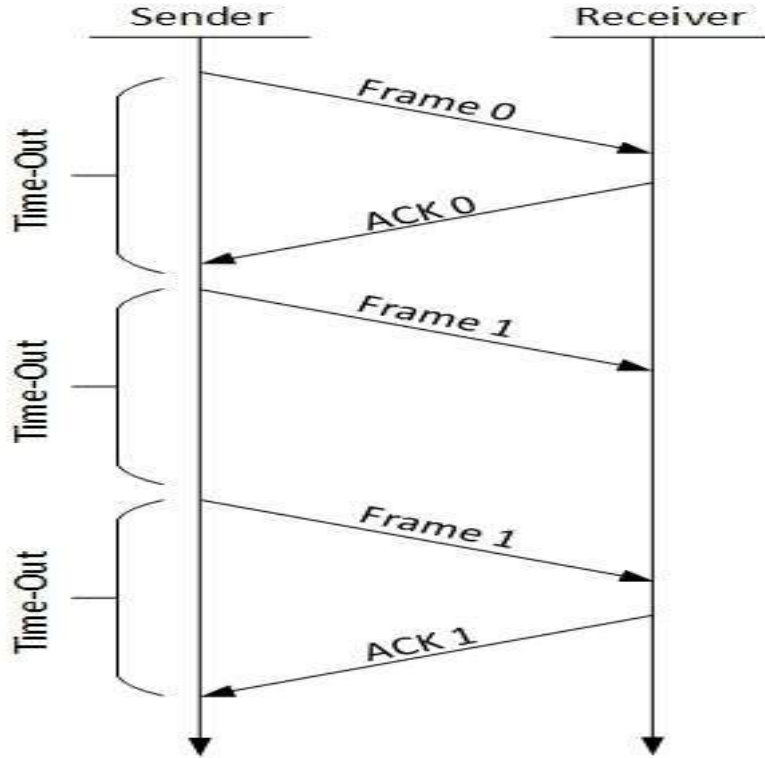
There are three main techniques for error control



Characteristics of Stop-and-Wait ARQ

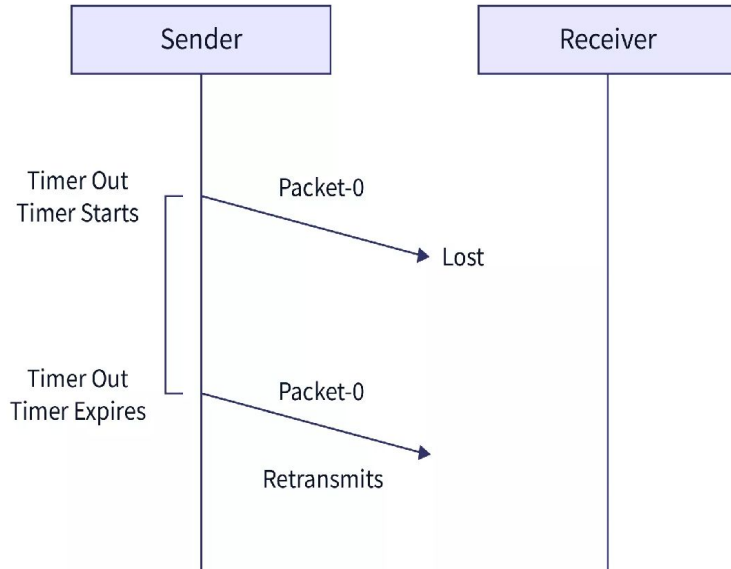
- The stop and wait ARQ is a sliding window protocol with a window size equal to 1.
- The stop and wait ARQ is an example of the connection-oriented.
- The sender sends the data frame with a sequence number.
- The sender also maintains a copy of the data frame that is being currently sent so that if the ACK is not received then the sender can re-transmit the frame.
- The sender can send only one frame at a time and the receiver can also receive only one frame at a time.
- In the stop and wait ARQ, the sender needs to maintain a time tracker.
- To detect corrupted frames, we need to add a CRC to each data frame. When a frame arrives at the receiver site, it is checked.

Stop-and-Wait Automatic Repeat Request

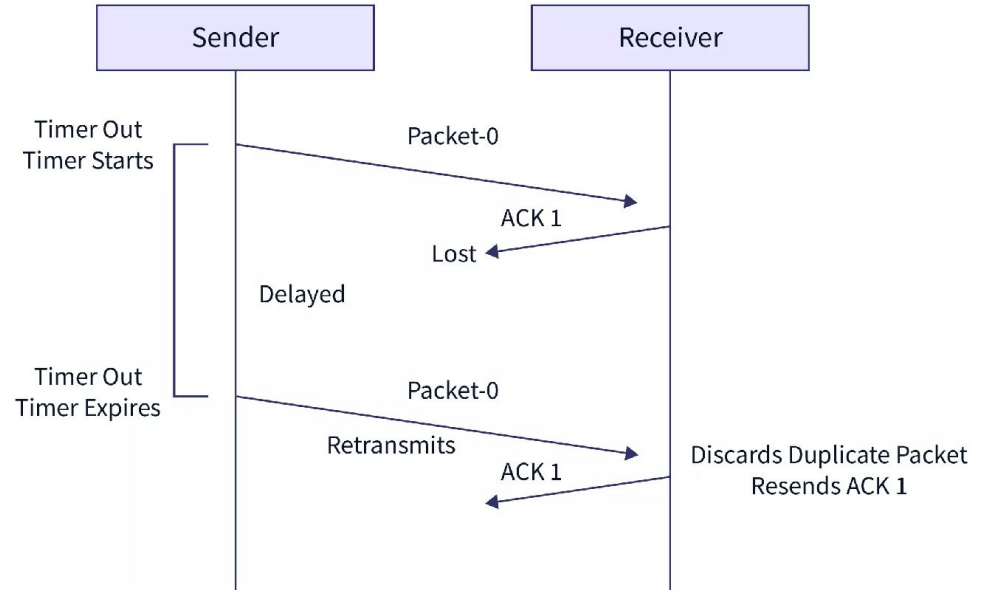


Stop-and-Wait Automatic Repeat Request

Problem of Lost Data Packet

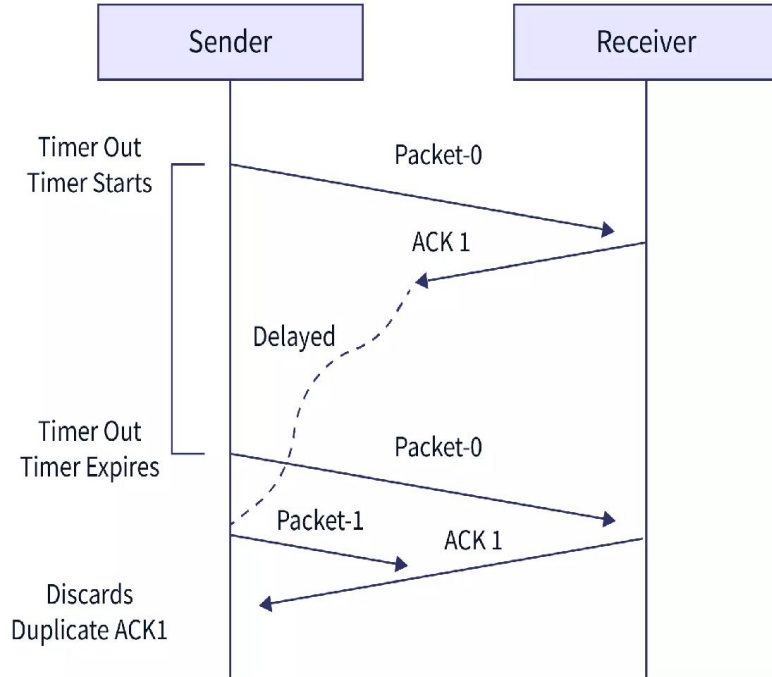


Problem of Lost Acknowledgement

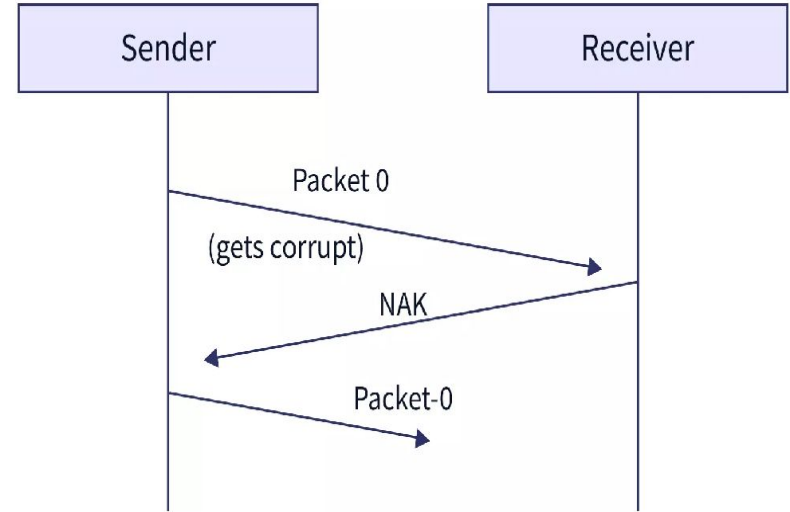


Stop-and-Wait Automatic Repeat Request

Problem of Delayed Acknowledgement



Problem of Damaged Packet



Advantages of Stop-and-Wait ARQ

- The stop and wait ARQ can be used in both the data link layer and the transport layer.
- The stop and wait ARQ provides both error management and flow control management.
- The stop and wait ARQ works in half-duplex mode as at a time only the sender can send the message or the receiver can send the ACK.
- The stop and wait ARQ ensures that the information is received by the receiver in the order it was sent.

Limitations of Stop-and-Wait ARQ

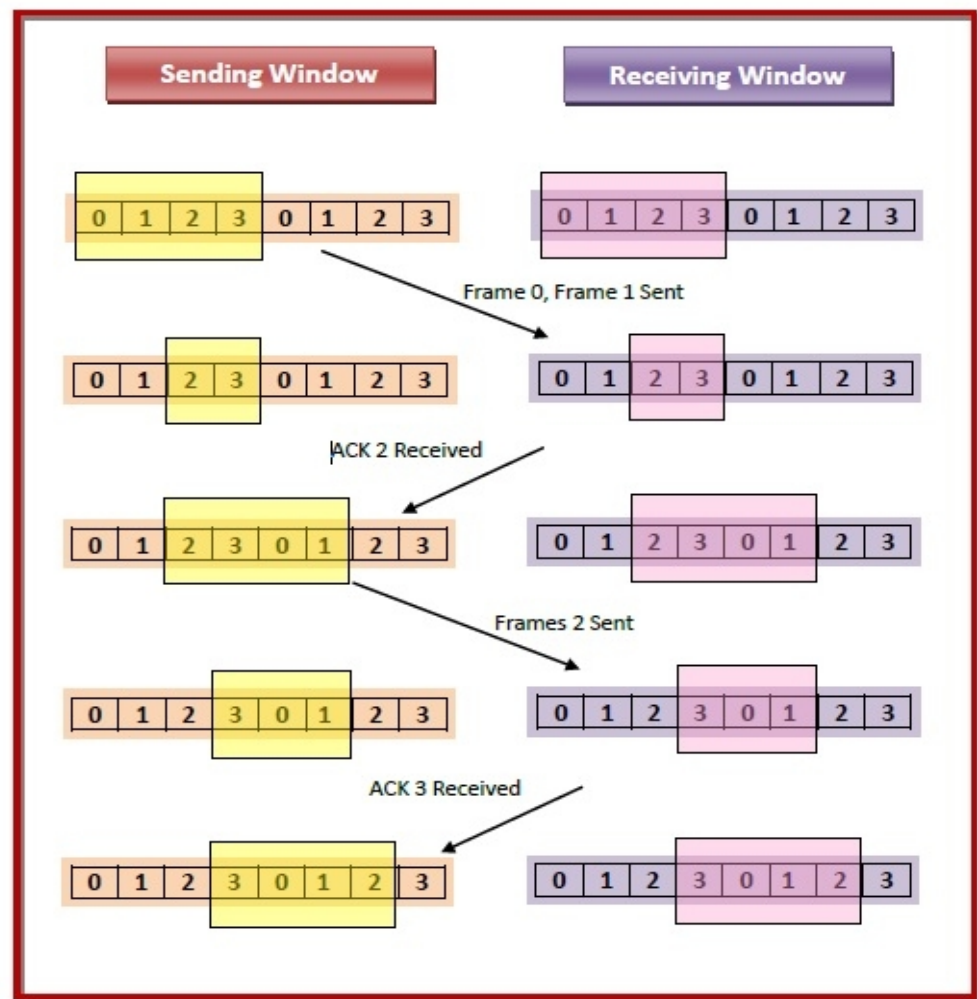
- The data can be lost in between the transmission. So, in such a case, the sender waits for ACK and the receiver waits for the data frame for an infinite amount of time.
- The ACK from the receiver may get lost in the channel. So, the sender waits for ACK for an infinite amount of time.
- The window size of the sender and the receiver is only 1. So, only one frame can be sent at a time.
- As there is a timer concept, so the sender must wait for a long duration before retransmission. Hence, the stop and wait ARQ is a slow protocol.

Sliding Window Protocol

- The sliding window is a technique for sending multiple frames at a time.
- It controls the data packets between the two devices where reliable and gradual delivery of data frames is needed
- In this technique, each frame has sent from the sequence number.
- The sequence numbers are used to find the missing data in the receiver end.
- The purpose of the sliding window technique is to avoid duplicate data, so it uses the sequence number.

Sliding Window Protocol

- Suppose that we have sender window and receiver window each of size 4.
- So the sequence numbering of both the windows will be 0,1,2,3,0,1,2 and so on.
- The diagram shows the positions of the windows after sending the frames and receiving acknowledgments.



Types of Sliding Window Protocol



Go-Back-N ARQ

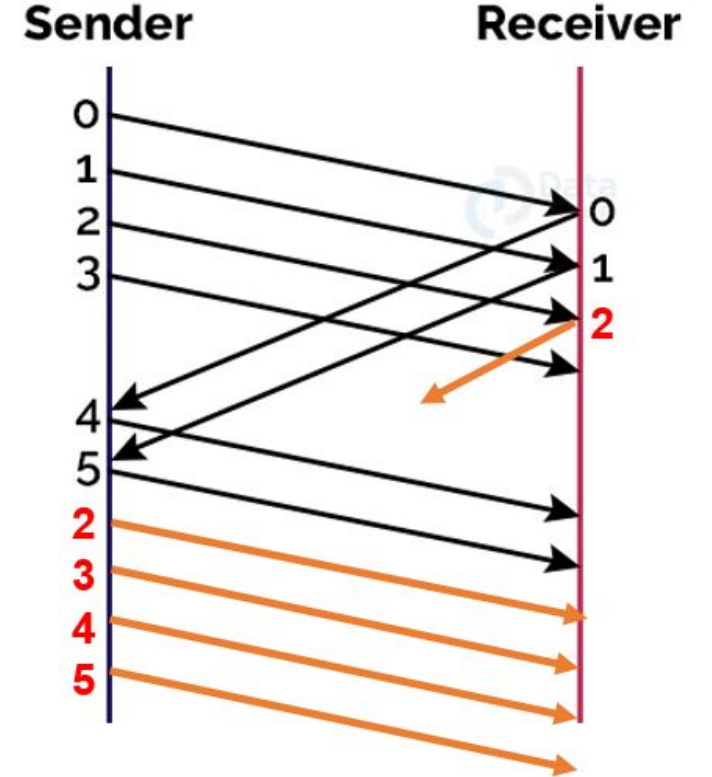
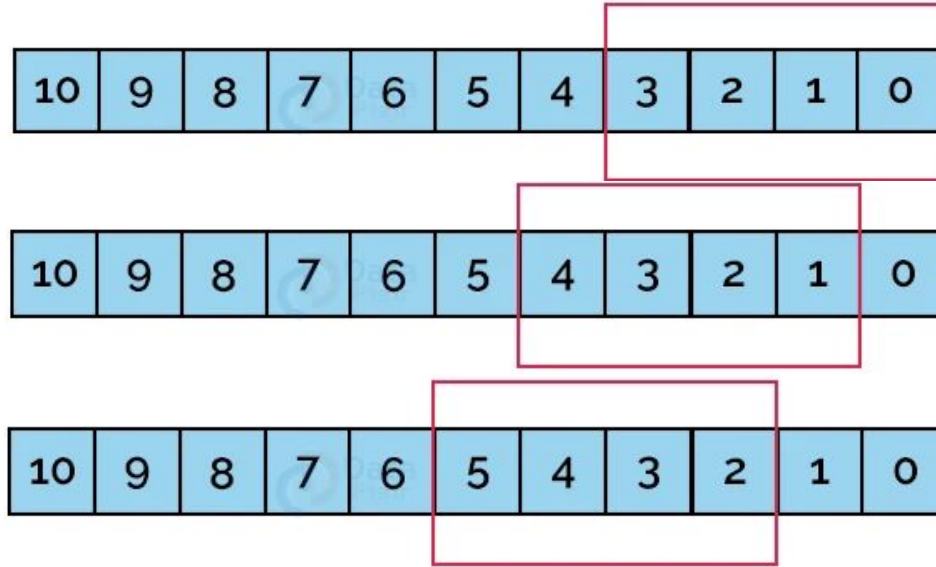
- Go-Back-N ARQ stands for Go-Back-N Automatic Repeat Request.
- It is a data link layer protocol that uses a sliding window method.
- Go-Back-N ARQ, we use the concept of a timer.
- When the receiver receives the correct frame, it sends back an acknowledgment or ACK.
- When sender gets an ACK for a data frame, it shifts its window forward and sends the next frame.
- If the ACK is not received within the specified period then all the data frames starting from the lost frame are retransmitted.
- The frames are sequentially numbered and a finite number of frames are sent.

Go-Back-N ARQ

- In these protocols, the sender has a buffer called the sending window and the receiver has buffer called the receiving window.
- The size of the sending window determines the sequence number of the outbound frames.
- If the sequence number of the frames is an n -bit field, then the range of sequence numbers that can be assigned is 0 to $2^n - 1$.
- Consequently, the size of the sending window is $2^n - 1$.
- For example, if the sending window size is 4, then the sequence numbers will be 0, 1, 2, 3, 0, 1, 2, 3, 0, 1, and so on. The number of bits in the sequence number is 2 to generate the binary sequence 00, 01, 10, 11.
- The size of the receiving window is the maximum number of frames that the receiver can accept at a time. The receiver's window size is always 1 for Go Back N ARQ.
- It determines the maximum number of frames that the sender can send before receiving acknowledgment.

Go-Back-N ARQ

.In Go Back 4 ARQ, the size of the sender window is 4.



Advantages of Go-Back-N ARQ

Advantages of Go-Back-N ARQ

1. It can send multiple frames at once.
2. Pipelining is present in the Go-Back-N ARQ i.e. a frame can be sent by the sender before receiving the acknowledgment of the previously sent frame. This results in a lesser waiting time for the frame.
3. It handles corrupted as well as out-of-order frames which result in minimal frame loss.

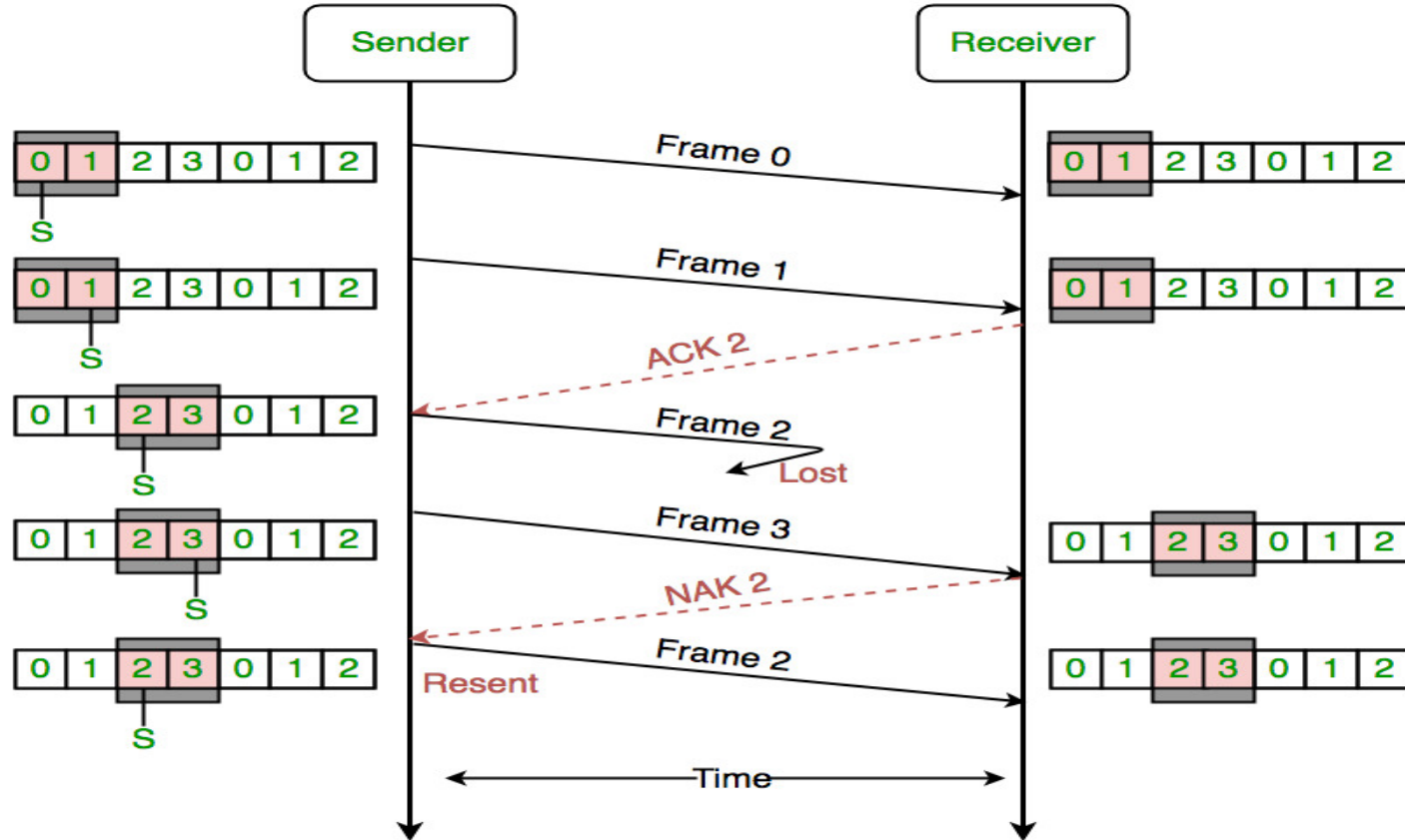
Disadvantages of Go-Back-N ARQ

1. If acknowledgment for a frame is not received, the whole window of frames is retransmitted instead of just the corrupted frame. This makes the Go Back N ARQ protocol inefficient.
2. Retransmission of all the frames on detecting a corrupted frame increases channel congestion and also increases the bandwidth requirement.
3. It is more time-consuming because while retransmitting the frames on detecting a corrupted frame, the error-free frames are also transmitted.

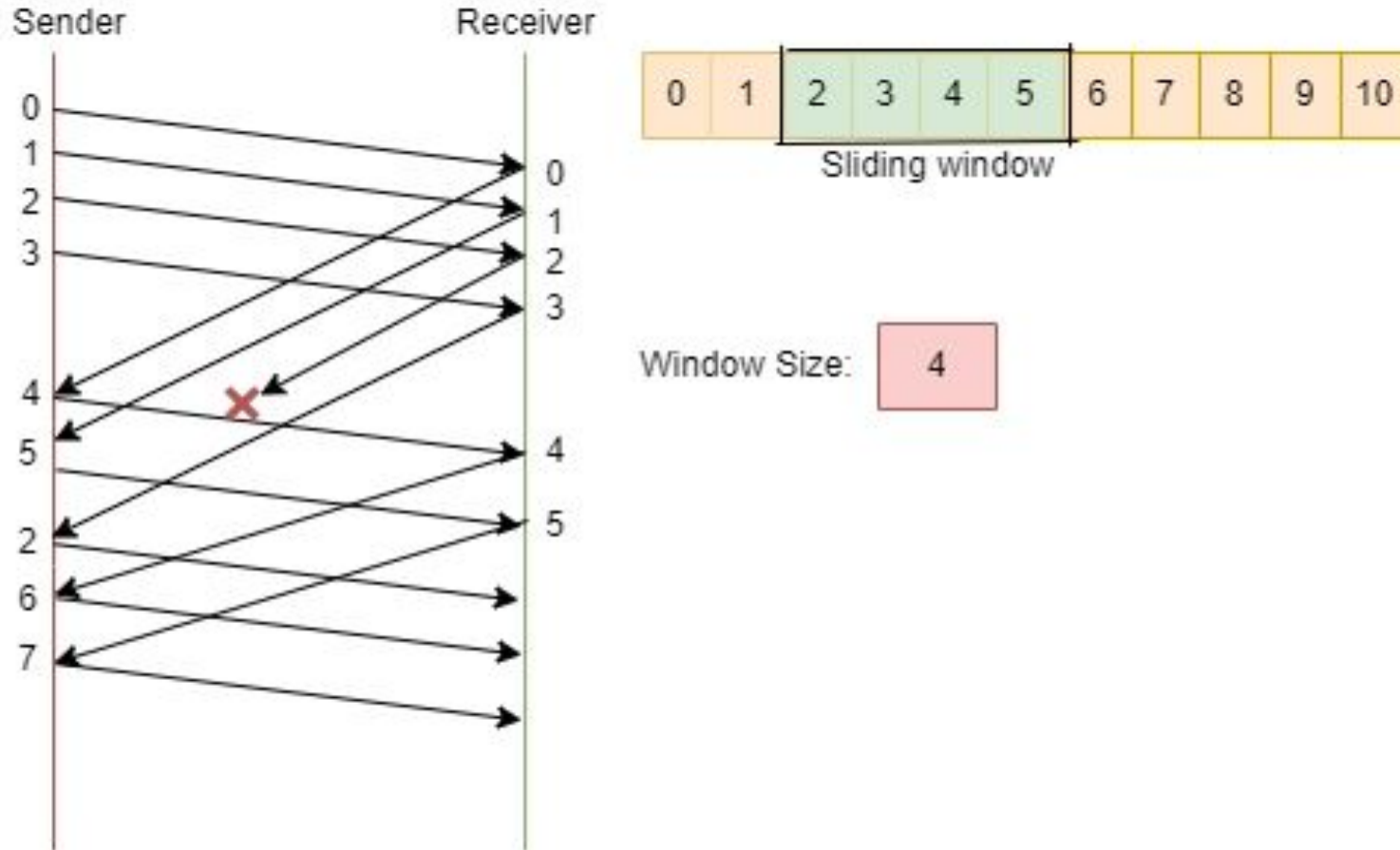
Selective Repeat ARQ

- The Go-back-N ARQ protocol works well if it has fewer errors.
- But if there is a lot of error in the frame, lots of bandwidth loss in sending the frames again.
- In the Selective Repeat ARQ protocol, the size of the sender window is always equal to the size of the receiver window. The size of the sliding window is always greater than 1.
- If the receiver receives a corrupt frame, it does not directly discard it.
- It sends a negative acknowledgment to the sender.
- The sender sends that frame again as soon as on the receiving negative acknowledgment.
- There is no waiting for any time-out to send that frame.

Selective Repeat ARQ



Selective Repeat ARQ



Advantages of Selective Repeat ARQ

- Efficient Data Transmission:
- Minimized Congestion:
- Faster Recovery
- Optimized Network Utilization:
- Improved Reliability:
- Supports High Error Rates:
- Adaptive:
- Higher Throughput:
- Minimal Latency:
- Efficient for Large Windows:

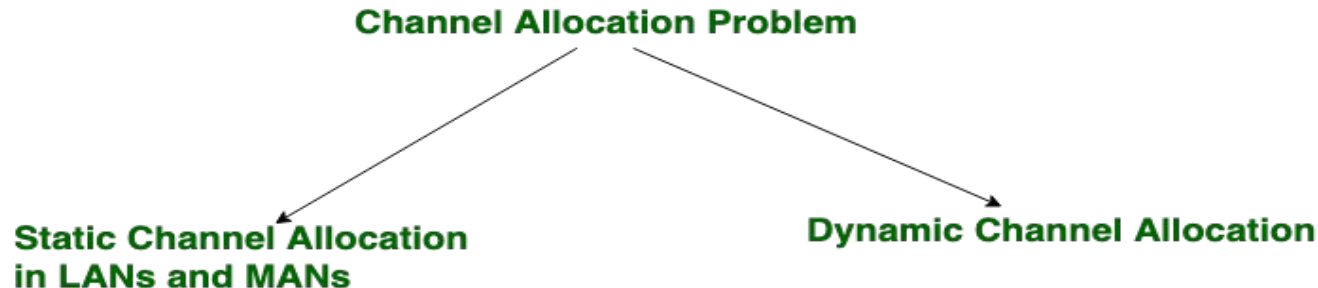
Disadvantages of Selective Repeat ARQ

- Increased Complexity:
- Higher Overhead:
- Out-of-Order Delivery:
- Retransmission Overhead:
- Increased Delay:
- Synchronization Dependency:

Go-Back-N ARQ	Selective Repeat ARQ
If a frame is corrupted or lost in it,all subsequent frames have to be sent again.	In this, only the frame is sent again, which is corrupted or lost.
If it has a high error rate,it wastes a lot of bandwidth.	There is a loss of low bandwidth.
It is less complex.	It is more complex because it has to do sorting and searching as well. And it also requires more storage.
It does not require sorting.	In this, sorting is done to get the frames in the correct order.
It does not require searching.	The search operation is performed in it.
It is used more.	It is used less because it is more complex.

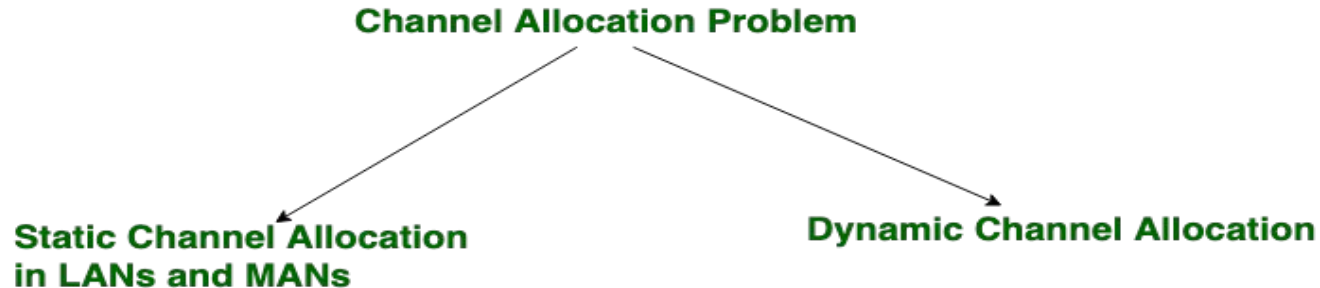
Channel Allocation Problem

- The channel allocation problem in the data link layer is the challenge of dividing and assigning a limited communication bandwidth and frequency spectrum among multiple devices/users
- to optimize network performance (maximize data throughput, minimize the delays) and
- prevent signal collisions or interference.



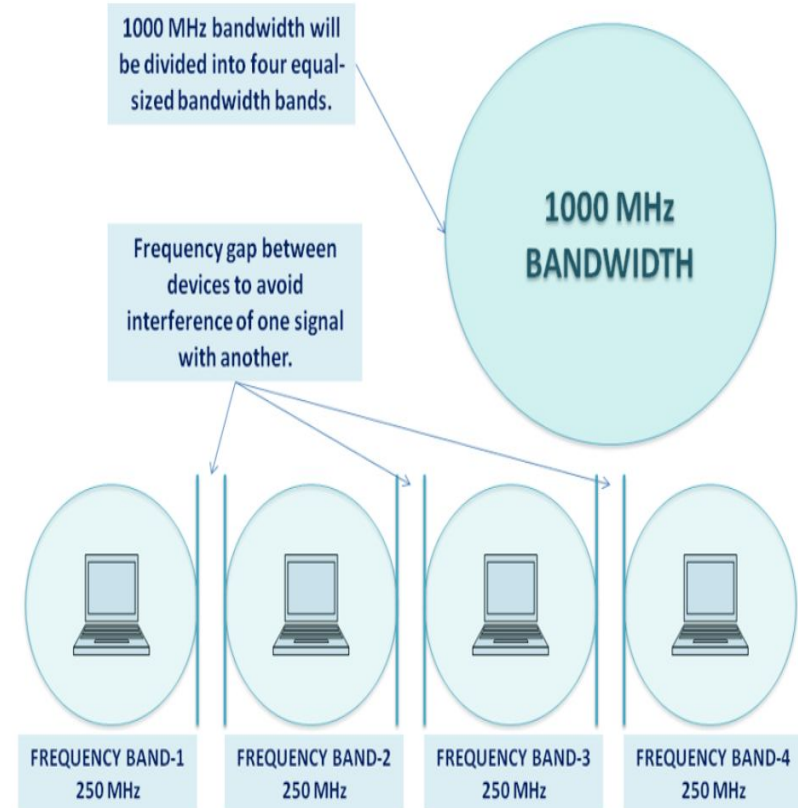
Channel Allocation Problem

- Channel Allocation Strategies are designed in such a way that there is efficient use of
- frequencies,
- time slots and
- bandwidth.



Static Channel Allocation

- Splits bandwidth into fixed, separate portions using method like FDM (Frequency Division Multiplexing)
- Assigns each user a dedicated slice of the channel.
- Zero interference /collisions, simple to design.
- Unused bandwidth cannot be given to active user, highly wasteful if user is idle.

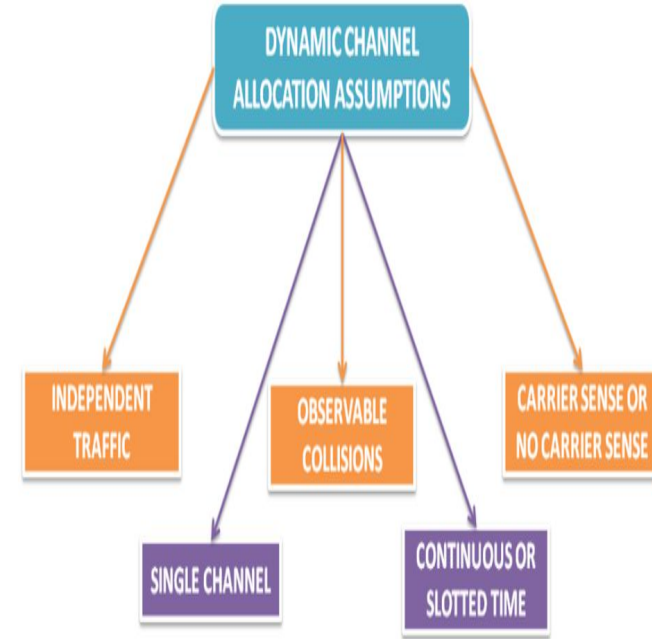


Dynamic Channel Allocation

- In dynamic channel allocation, the frequency bands are not permanently assigned to the users,
- Instead channels are allotted to users dynamically as needed.
- When the device completes its communication on a channel, it is de-allocated, and the same channel can be assigned to another device.

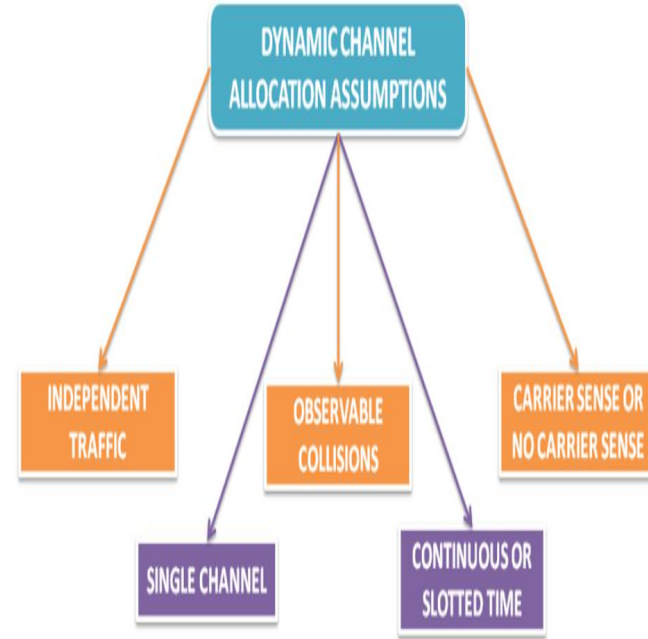
Assumptions of Dynamic Channel Allocation

- There are n **independent devices** controlled by the program/user that generate frames for transmission.
- In **single-channel** assumption, protocols are used to prioritize frames from less important frames to more important frames.
- The **collision** occurs when two frames are traveling on a shared channel and overlap with each other. When a collision occurs between frames, all devices can detect that a collision occurred on a channel.



Assumptions of Dynamic Channel Allocation

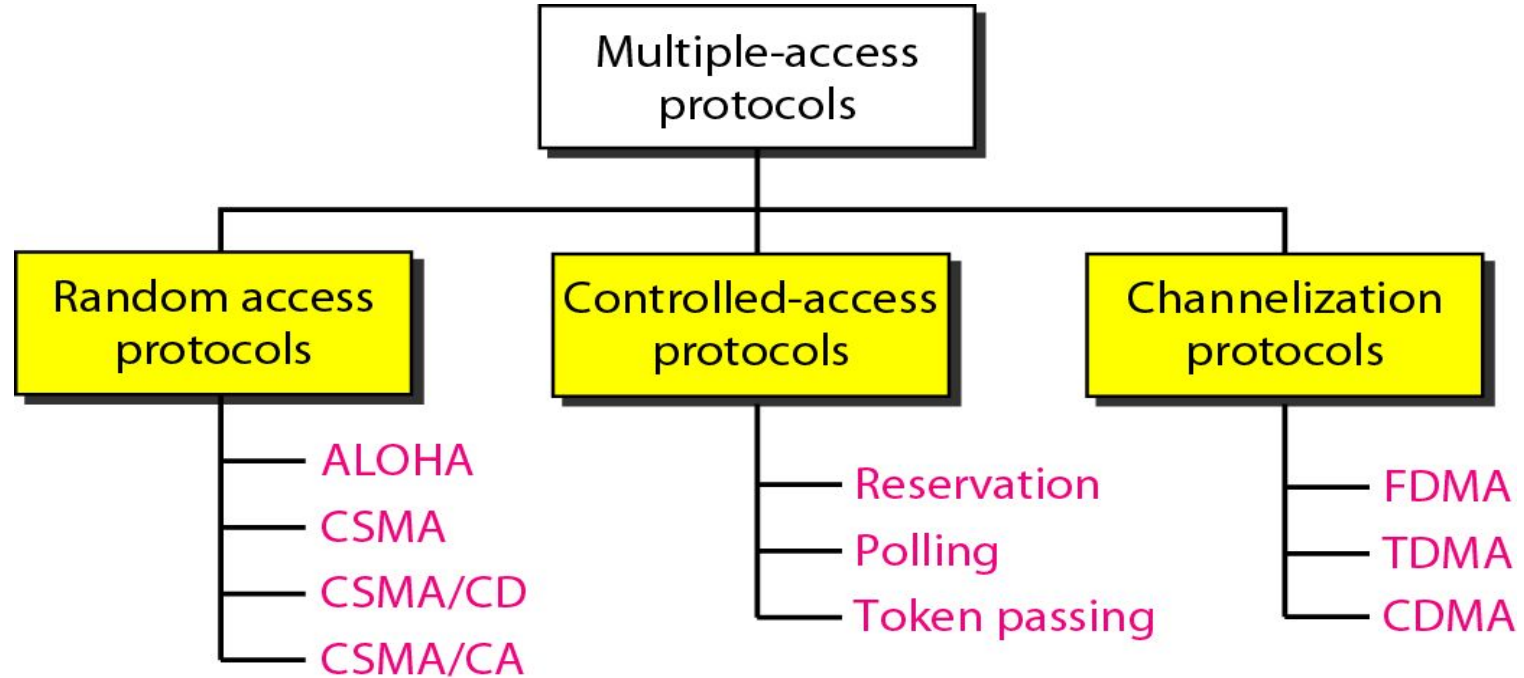
- We can consider the **time constant** as when frame transmission can be initiated at any instant.
- Time can be slotted or divided into discrete intervals known as slots. When the slot is allocated to the device or channel, the frame transmission must start at the beginning of the slot.
- In the **carrier** sense, the device checks whether the channel is in use before using it.
- If the station finds the channel busy, it will not broadcast any frame on the channel.
- If there is no carrier sense, the station may not understand the channel before use, leading to frame collisions.



Multiple Access Protocol

- The **data link layer** is used in a computer network to transmit the data between two devices or nodes.
- It divides the layer into parts such as **data link control** and the **multiple access resolution/protocol**.
- The upper layer has the responsibility to flow control and the error control in the data link layer, and hence it is termed as **logical of data link control**.
- Whereas the **lower sub-layer is used to handle and reduce the collision or multiple access on a channel**.
- Hence it is termed as **media access control** or the multiple access resolution/protocol.

Multiple Access Protocols



Random Access Protocol

- No station is superior to another station and none is assigned the control over another.
- No station permits, or does not permit, another station to send.
- There is no fixed time for sending data
- There is no fixed sequence of stations sending data
- Protocols
 - ALOHA (Pure and Slotted ALOHA)
 - CSMA
 - CSMA/CD
 - CSMA/CA

ALOHA

It is designed for wireless LAN (Local Area Network) but can also be used in a shared medium to transmit data.

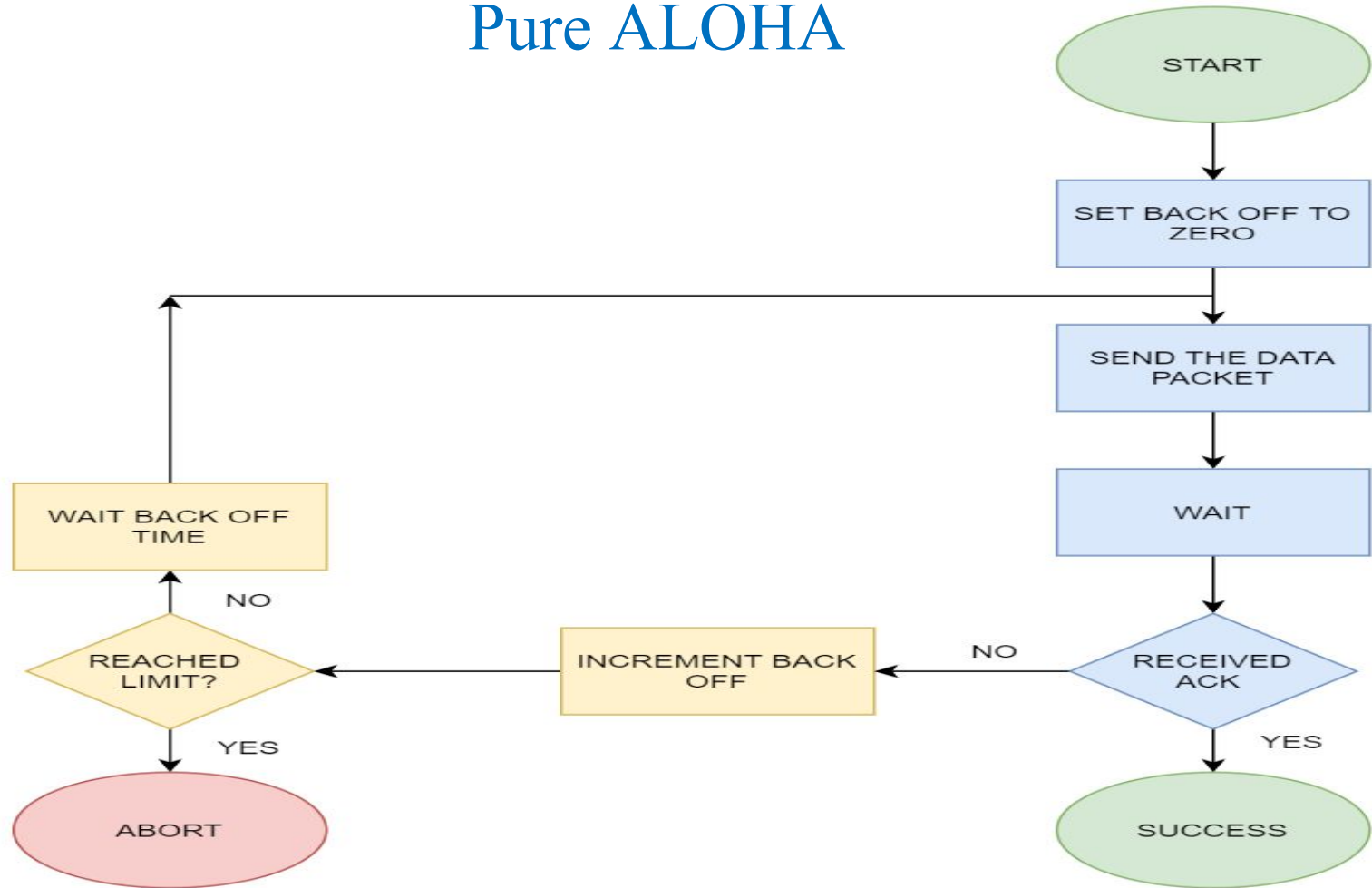
Aloha Rules

1. Any station can transmit data to a channel at any time.
2. It does not require any carrier sensing.
3. Collision and data frames may be lost during the transmission of data through multiple stations.
4. Acknowledgment of the frames exists in Aloha. Hence, there is no collision detection.
5. It requires retransmission of data after some random amount of time.

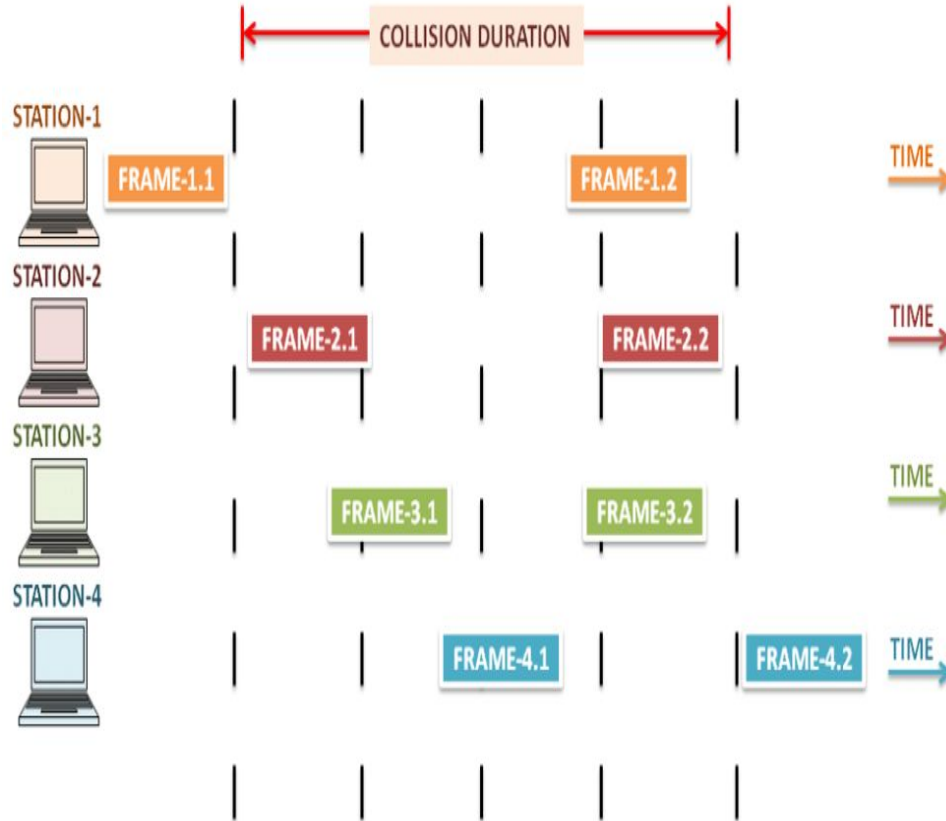
Pure ALOHA

- In pure Aloha each station transmits data to a channel without checking whether the channel is idle or not
- The chances of collision may occur, and the data frame can be lost.
- When any station transmits the data frame to a channel, the pure Aloha waits for the receiver's acknowledgment.
- If it does not acknowledge the receiver end within the specified time, the station waits for a random amount of time, called the backoff time (T_b).
- The station may assume the frame has been lost or destroyed.
- Therefore, it retransmits the frame until all the data are successfully transmitted to the receiver.

Pure ALOHA



Pure ALOHA

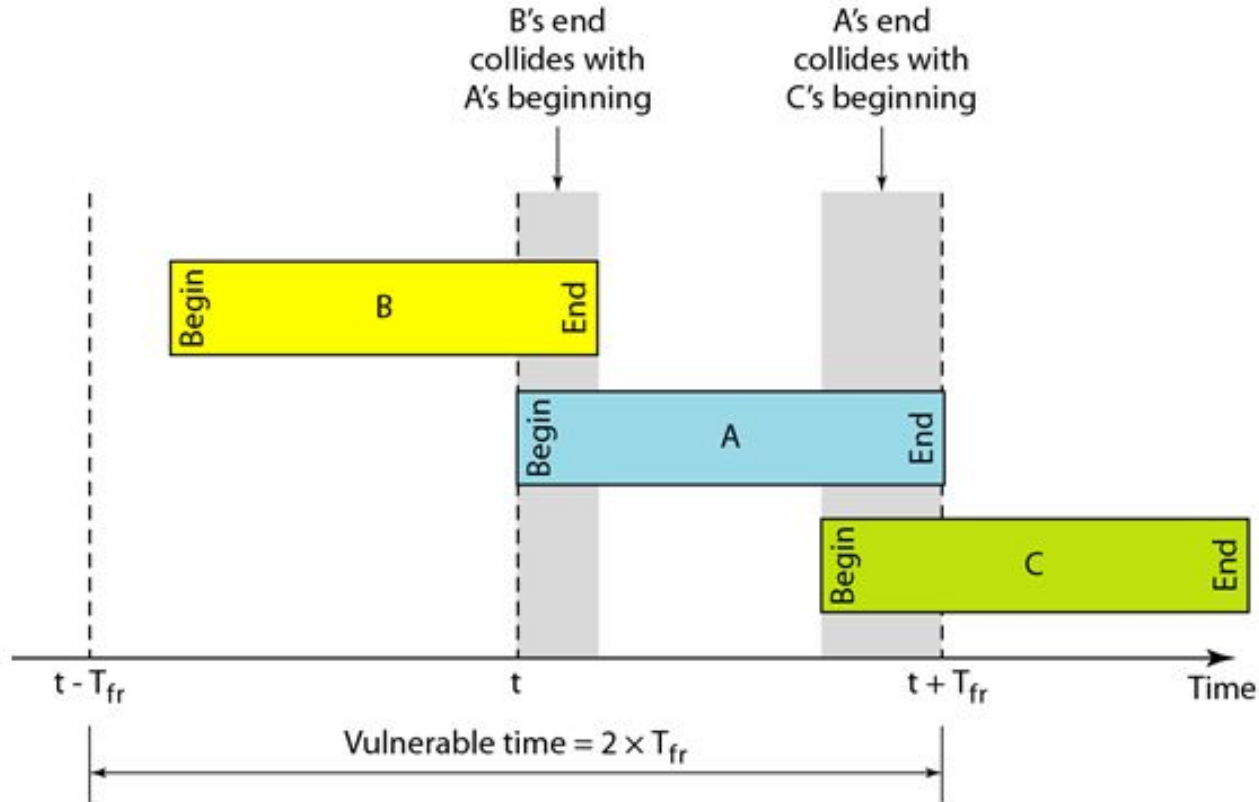


- 4 devices connected on a **shared channel**.
- Only frame-1.1 and frame-4.2 survive and reach the destination, while other frames are lost during transmission due to collisions with each other.
- When a receiver device successfully receives a frame, it sends an **acknowledgment** to the sending device.
- If the acknowledgment is not received after the time-out period, the sender assumes that the frame has been destroyed and resends the frame.
- After the timeout, collisions between frames occur when all devices resend the lost frame at once.
- To avoid collisions, each device waits a random amount of time before resending its frame.
- Random timing helps to avoid more collisions known as **back-off time T_b** .

Pure ALOHA

- In this method, there are maximum **K-max** attempts to retransmit the frame. The value of K-max is usually 15.
- If the device retransmits the frame for **K-max** times, the device should give up and try again.
- Here the back-off time T_b depends on the number of failed transmissions, which is **K**.
- The time taken to transmit a frame is known as Transmission Time (**T_{fr}**). In Pure ALOHA, the vulnerable time is twice the transmission time.

Vulnerable period for Pure ALOHA



Pure ALOHA

- The throughput of pure ALOHA can be calculated as,
 - $S = G \times e^{(-2G)}$
 - Where S = average number of successful transmissions for pure ALOHA,
 - G = the average number of frames generated by the device during transmission time of one frame.
 - Pure ALOHA has maximum throughput **S-max = 0.184 when G = (1/2)**

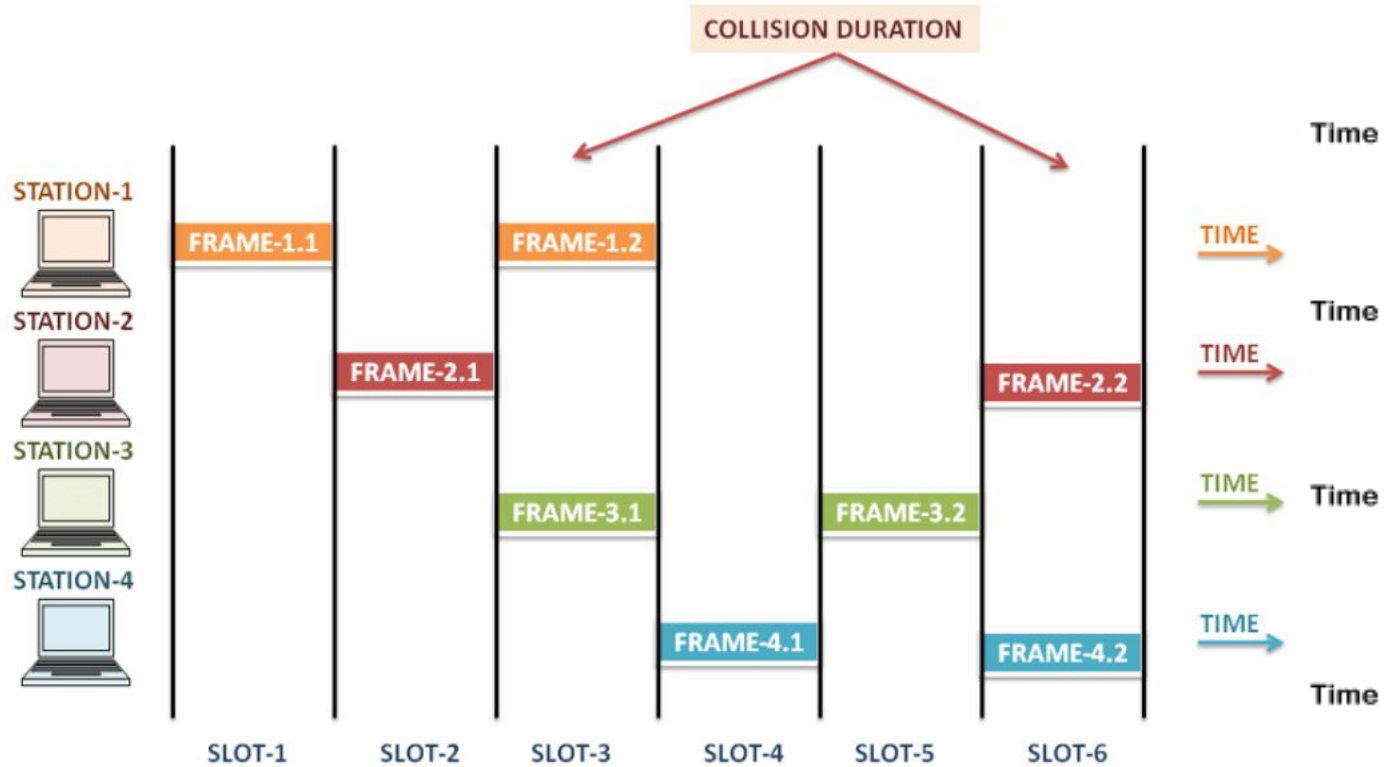
Slotted ALOHA

In Pure Aloha, no rules determine when a device can send frames over a channel. A device can send frames on the channel in two conditions. First, immediately after the transmission of another device has started. Second, immediately before the transmission of another device begins. But this increases the chances of a collision. So, slotted ALOHA solves this problem and is used to improve the efficiency of pure ALOHA.

Slotted ALOHA

- In slotted ALOHA, time is divided into T_{fr} slots, so the device can only send at the beginning of the time slot.
- The device is only allowed to send at the beginning of the synchronized time slot. If it misses, it will have to wait until the start of the next time slot.
- If two devices try to send at the beginning of the same time slot, the possibility of a collision between the devices still exists.

Slotted ALOHA

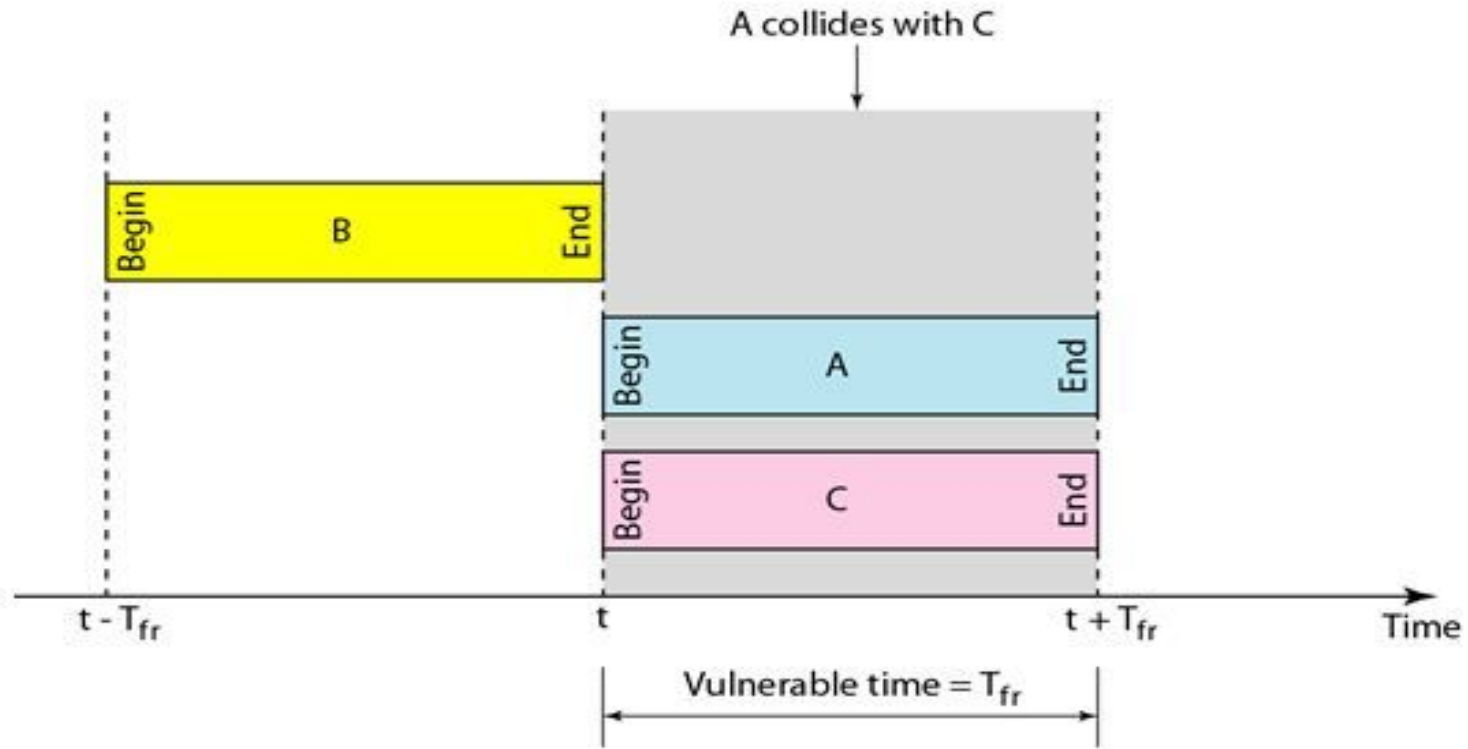


Frames in Slotted ALOHA

Slotted ALOHA

- As shown in the figure, four devices shared a channel. Each device is sending two frames on a single channel.
- Here the collisions are less as compared to pure ALOHA because they send the frame only when the particular time slot starts.
- Out of 8 frames, 4 frames (Frame-1.1, Frame-2.1, Frame-4.1, and Frame-3.2) were successfully transmitted, and other frames were lost during transmission.
- Slotted ALOHA vulnerable time is half that of pure ALOHA.
- In slotted ALOHA, throughput can be measured as follows:
 - $S = G \times e^{-G}$
 - Slotted ALOHA has maximum throughput **S-max = 0.368 when G = 1.**

Vulnerable period for Slotted ALOHA

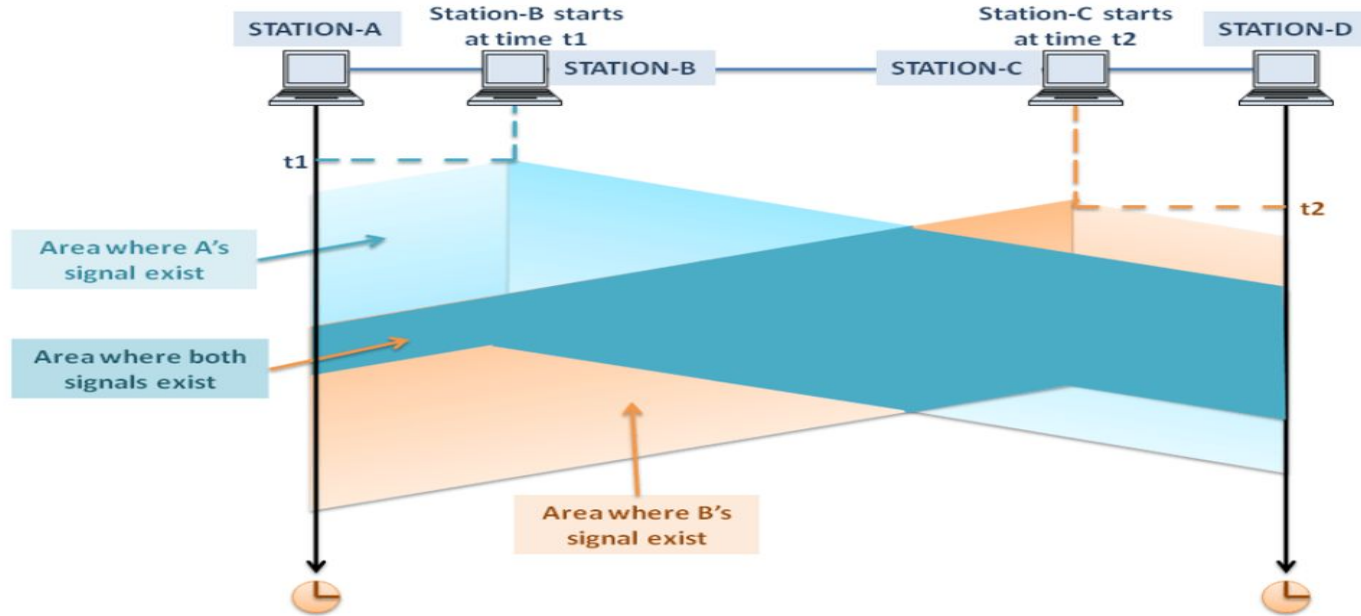


Key	Pure Aloha	Slotted Aloha
Time Slot	In Pure Aloha, any station can transmit data at any time.	In Slotted Aloha, any station can transmit data only at the beginning of a time slot.
Time	In Pure Aloha, time is continuous and is not globally synchronized.	In Slotted Aloha, time is discrete and is globally synchronized.
Vulnerable time	The vulnerable time or susceptible time in Pure Aloha is equal to $(2 \times T_t)$.	In Slotted Aloha, the vulnerable time is equal to (T_t) .
Probability	The probability of successful transmission of a data packet $(\eta) = G \times e^{-2G}$	The probability of successful transmission of a data packet $(\eta) = G \times e^{-G}$
Maximum efficiency	Maximum efficiency = 18.4%.	Maximum efficiency = 36.8%.
Number of collisions	Does not reduce the number of collisions.	Slotted Aloha reduces the number of collisions to half, thus doubles the efficiency.

Carrier Sense Multiple Access(CSMA)

- It is a **carrier sense multiple access** based on media access protocol to sense the traffic on a channel (idle or busy) before transmitting the data.
- It means that if the channel is idle, the station can send data to the channel. Otherwise, it must wait until the channel becomes idle.
- Hence, it reduces the chances of a collision on a transmission medium.
- CSMA method was developed to decrease the chances of collisions when 2 or more stations start sending their signals over the data link layer

Carrier Sense Multiple Access(CSMA)



Here the time required for the propagation of the signal from one end of the channel to the other is known as a propagation or vulnerable time .

Carrier Sense Multiple Access(CSMA)

But, what to do if the channels are busy? Now, here the persistence methods can be applied to help the station act when the channel is busy or idle.

CSMA uses the “**sense before transmit**” or “**listen before talk**” principle, allowing the device to sense the channel before using it.

Carrier Sense Multiple Access

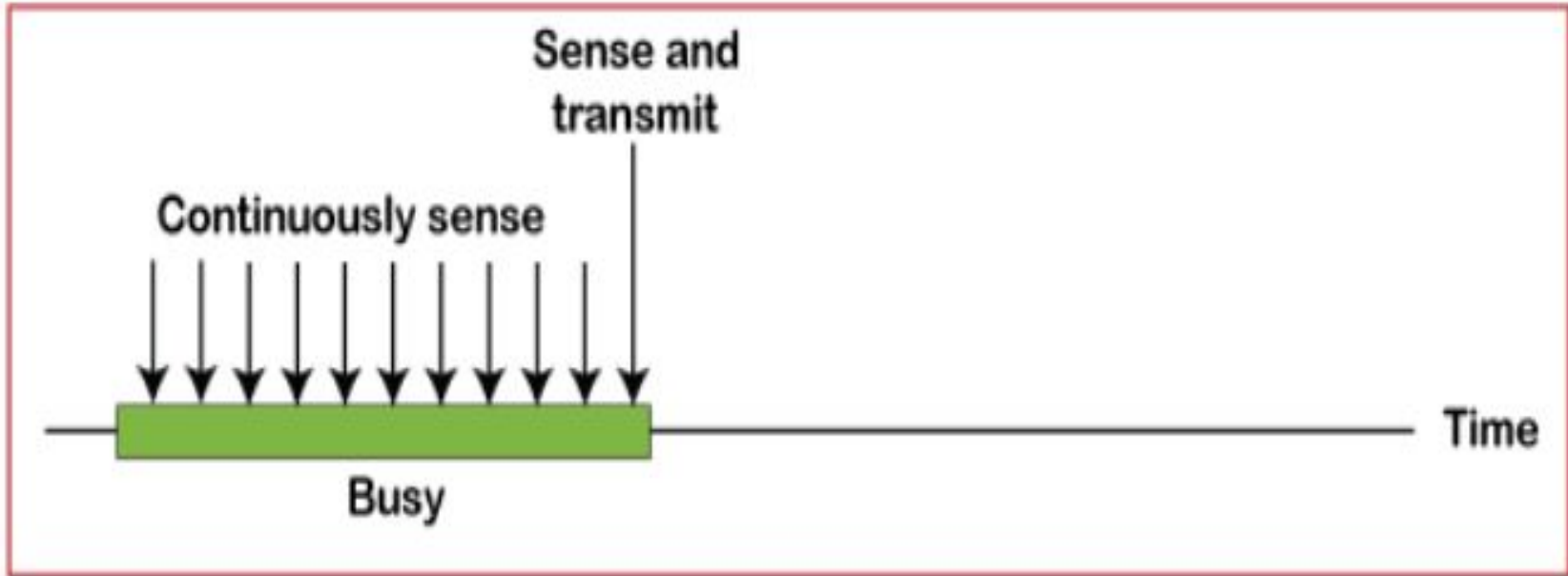
The CSMA has 4 access modes:

- **1-persistent mode**
- **Non-persistent mode**
- **P-persistent mode**
- **O-persistent**

• **1-persistent mode:** In this, first the node checks the channel, if the channel is idle then the node or station transmits data, otherwise it keeps on waiting and whenever the channel is idle, the stations transmit the data-frame.

Carrier Sense Multiple Access (CSMA)

1-persistent mode

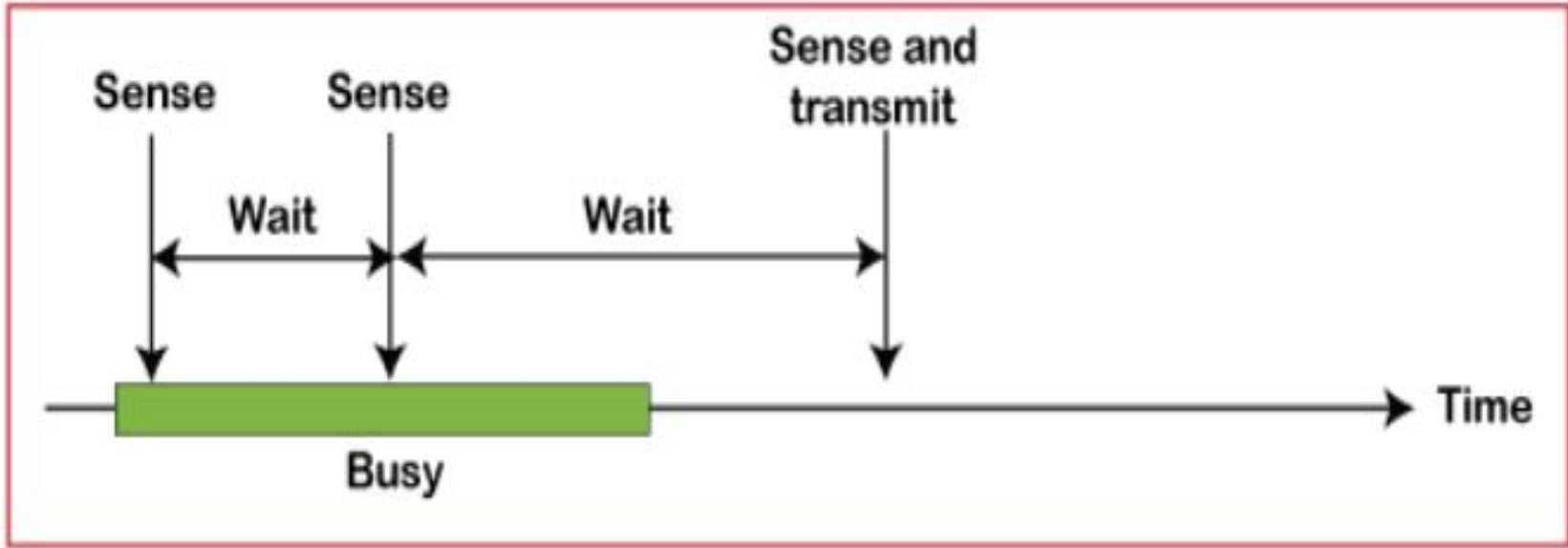


CSMA protocols

- **Non-persistent mode:** In this, the station checks the channel similarly as 1-persistent mode, but the only difference is that when the channel is busy it checks it again after a random amount of time, unlike the 1-persistent where the stations keep on checking continuously.

Carrier Sense Multiple Access (CSMA)

Non-persistent mode



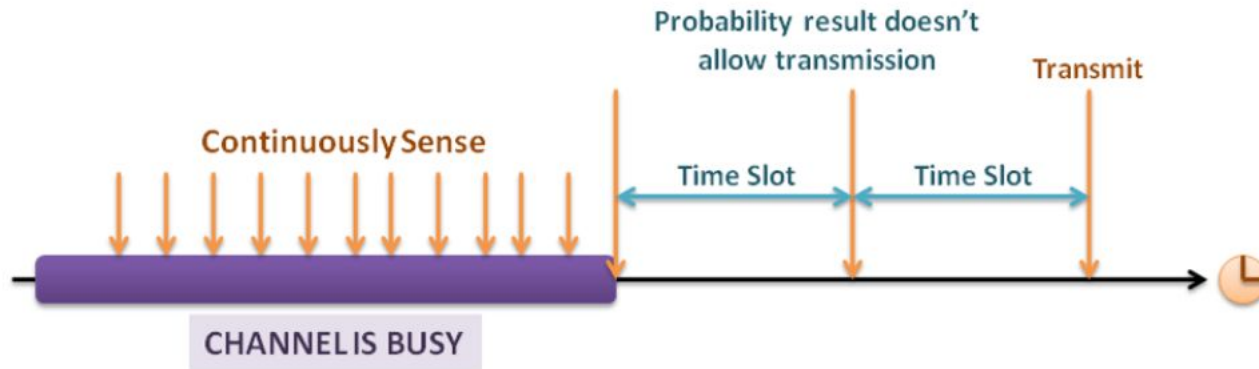
CSMA protocols

P-persistent mode: The station sends a frame with probability p . The station waits for the next time slot to start and senses the channel again with probability $q = 1-p$. If it finds the channel idle, it sends the frame. Otherwise, it acts as if a collision has occurred and uses the back-off procedure.

Carrier Sense Multiple Access (CSMA)

P-persistent mode

- As shown in the figure, the station senses the channel continuously. When the station finds the channel idle, it checks the probability result.
- If the probability result is greater than the probability p , it waits for a slot and senses the channel again. If it finds the channel busy, it uses the back-off procedure, though a collision occurred. Otherwise, it checks the probability result.
- If the probability result is less than or equal to p , the station will send the frame.



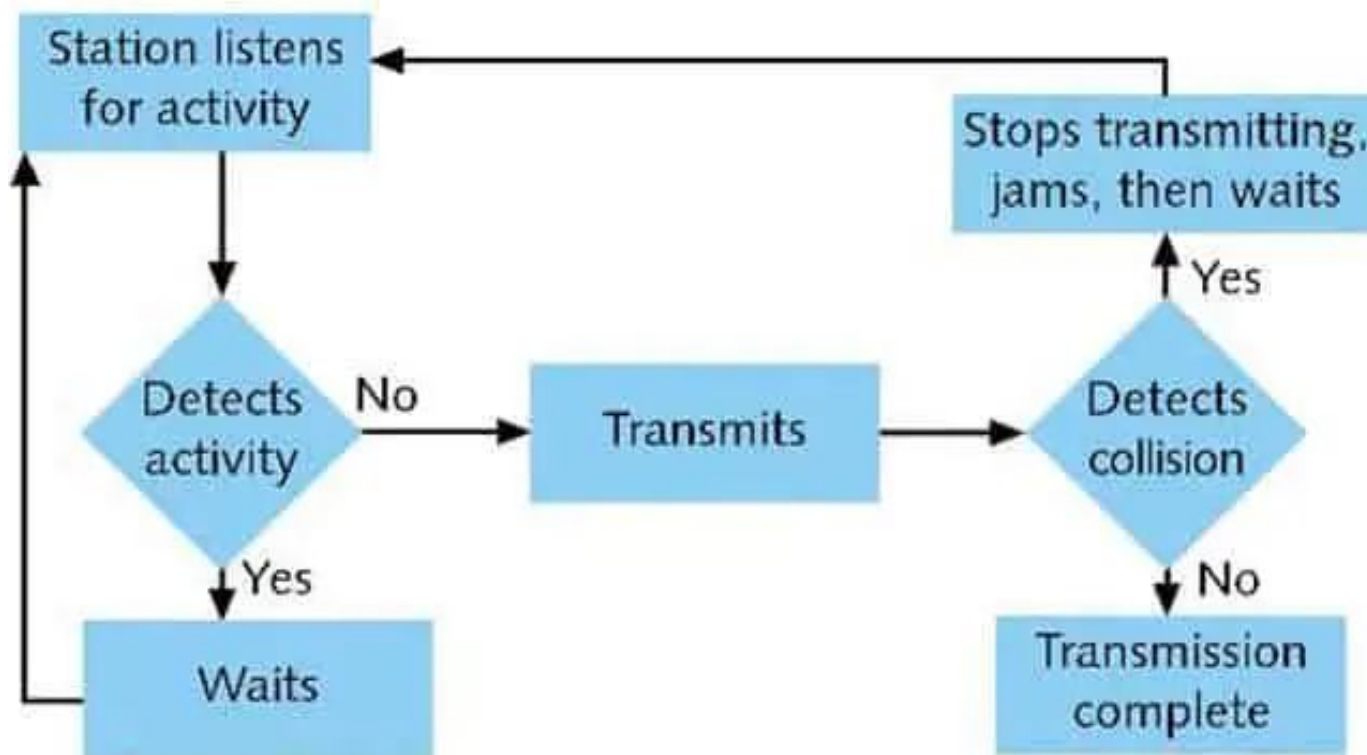
CSMA protocols

- O-persistent:** In this, the transmission occurs based on the superiority of stations which is decided beforehand and transmission occurs in that order. If the channel is idle, then the station waits for its turn to send the data-frame.

CSMA with Collision Detection

- It is a **carrier sense multiple access/ collision detection** network protocol to transmit data frames.
- The CSMA/CD protocol works with a medium access control layer.
- Therefore, it first senses the shared channel before broadcasting the frames, and if the channel is idle, it transmits a frame to check whether the transmission was successful.
- If the frame is successfully received, the station sends another frame.
- If any collision is detected in the CSMA/CD, the station sends a jam/ stop signal to the shared channel to terminate data transmission.
- After that, it waits for a random time before sending a frame to a channel.

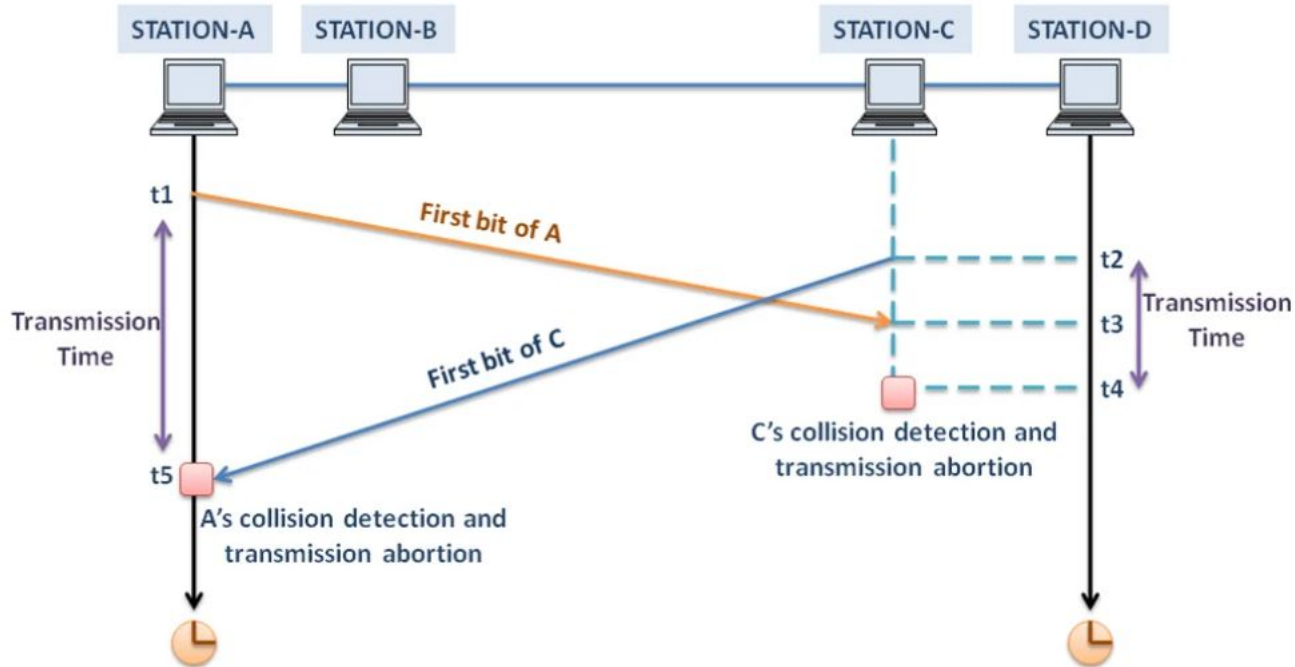
CSMA with Collision Detection



CSMA with Collision Detection

- CSMA/CD uses persistence methods to understand the channel. Transmission and collision detection is a continuous process in CSMA/CD.
- In CSMA/CD, the station transmits and receives frames continuously and simultaneously.
- The station continuously monitors the channel to see if a broadcast has ended or a collision is detected on a channel. If the collision is not detected, it means that the transmission is complete.
- The maximum throughput of CSMA/CD depends on the value of G and the persistence methods. Its throughput value is higher than that of Aloha's two protocols.
- For the 1-persistent and non-persistent methods, the maximum throughput is approximately **50% ($G=1$) and 90% (between $G=3$ and 8)**, respectively.

CSMA with Collision Detection



CSMA/CA

- In CSMA/CD, when there is a collision on the channel, the station gets to know about it.
- It receives its own signal when there is no collision and receives its own and another station's signal when a collision is detected.
- CSMA/CA was invented to avoid collisions over a wireless channel, as wireless networks cannot detect collisions

CSMA/CA

Collision is avoided by using three strategies of CSMA/CA.

1. Inter-Frame Space (IFS)
2. Contention Window
3. Acknowledgments

CSMA/CA

Inter-Frame System: When a station finds the channel idle, it does not transmit the frame immediately. It waits for some time which is known as inter-frame space or IFS.

- When the channel is idle after the IFS time, the station sends the frame after waiting for some time, equal to the contention time.
- IFS is used to define the priority of a station on a channel. For example, a station with a short IFS has the highest priority among all stations.

CSMA/CA

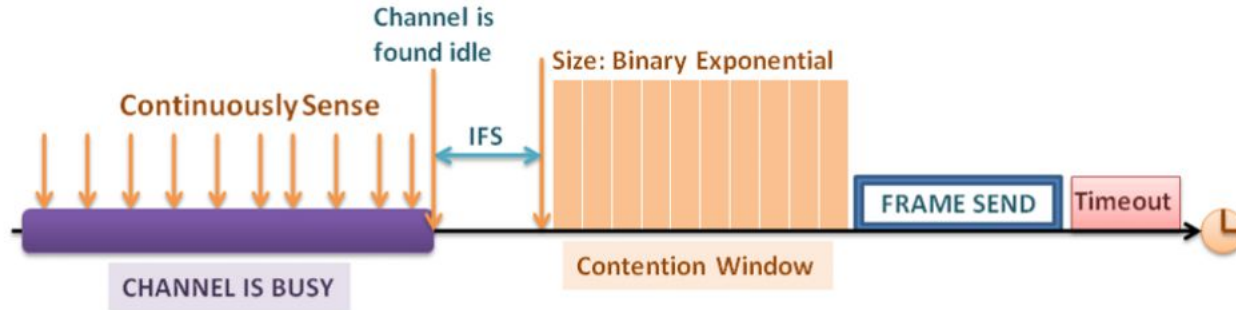
Contention Window: The contention window is the time that comes immediately after IFS. It is divided into smaller time slots to prevent multiple accesses on a single channel between devices. The station ready to send the frame chooses a random number of slots as its waiting time.

- The contention window is first set to one slot, and if the station cannot detect an idle channel after IFS time, the slot in the window is doubled each time.
- The contention window is somehow similar to the P-persistent scheme, but here on a channel, the random outcome defines the total slots occupied by the waiting station.
- Station senses the channel after each time slot in the contention window. If the station finds the channel busy, it does not resume the processing of the contention window. It stops the timer and senses the channel. If the station finds the channel idle, it restarts the timer.

CSMA/CA

Acknowledgments: Collisions can occur on a single channel resulting in loss of frames. So, the receiver uses the acknowledgment to acknowledge to the sender that it has received the frame. The positive acknowledgment and time-out clock guarantee that the receiver has received the frame sent by the sender.

CSMA/CA



- As shown in the figure, the channel is sensed by the station before and after IFS. The station also senses the channel during contention time and for each time slot of the contention window.
- In the idle channel, the timer is continuous. If the channel is busy, the channel timer stops, and the channel continues after the timer becomes idle again.

Text Books:

1. Behrouz A. Forouzan, 'Data Communications and Networking', 5th Edition, McGraw-Hill Publishing Company, ISBN 978-0-07-337622-6
2. Tanenbaum A. S., 'Computer Networks', Pearson Education , 5th Edition, ISBN-978-0-13-212695-3

Reference Books:

1. James F. Kurose and Keith W Ross 'Computer Networking, A Top-Down Approach', 5th Edition, Pearson Education, ISBN-978-81-317-9054-0
2. W. Richard Stevens, Unix Network Programming, The Sockets Networking API, Vol 1, 3rd Edition, PHI Learning Pvt. Ltd.

Supplementary Reading:

1. William Stallings, 'Data and Computer Communications', 6th Edition, Prentice Hall of India Pvt.

Web Resources:

1. <https://nptel.ac.in/courses/106/105/106105080/>
2. <https://nptel.ac.in/courses/106/106/106106091/>

Web links:

1. <https://nptel.ac.in/courses/106/105/106105081/>
2. <https://nptel.ac.in/courses/106/105/106105183/>
3. <https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-829-computer-networks-fall-2002/index.htm>

MOOCs:

1. <https://www.coursera.org/learn/computer-networking>
2. <https://www.edx.org/course/introduction-to-networking?index=product&queryID=0befdc538866babf2facfdc2dce06204&position=1>

